UNIVERSIDADE REGIONAL DE BLUMENAU

CENTRO DE CIÊNCIAS EXATAS E NATURAIS

CURSO DE CIÊNCIA DA COMPUTAÇÃO – BACHARELADO

VISUALIZADOR DE IMAGENS 360 GRAUS INTERATIVAS NA PLATAFORMA ANDROID

JORGE LUIS ITEN JÚNIOR

BLUMENAU
2012

2012/1-17

JORGE LUIS ITEN JÚNIOR

VISUALIZADOR DE IMAGENS 360 GRAUS INTERATIVAS NA PLATAFORMA ANDROID

Trabalho de Conclusão de Curso submetido à Universidade Regional de Blumenau para a obtenção dos créditos na disciplina Trabalho de Conclusão de Curso II do curso de Ciência da Computação — Bacharelado.

Prof. Dalton Solano dos Reis, Mestre – Orientador

BLUMENAU
2012

2012/2-17

VISUALIZADOR DE IMAGENS 360 GRAUS INTERATIVAS NA PLATAFORMA ANDROID

Por

JORGE LUIS ITEN JÚNIOR

Trabalho aprovado para obtenção dos créditos na disciplina de Trabalho de Conclusão de Curso II, pela banca examinadora formada por:

Presidente:	Prof. Dalton Solano dos Reis, Mestre – Orientador, FURB
Membro:	Prof. Aurélio Faustino Hoppe, Mestre – FURB
Membro:	Prof. Miguel Alexandre Wisintainer, Mestre – FURB

Blumenau, 10 de julho de 2012

Dedico este trabalho a toda minha família que tornou possível a realização deste.

AGRADECIMENTOS

Aos meus pais, pelo apoio e principalmente a dedicação na minha educação e que permitiram que eu pudesse chegar até aqui. Sem eles, com toda a certeza eu não teria chegado nem perto da finalização deste trabalho. Também ao meu irmão que me ajudou a optar por este curso.

A minha namorada Camila pela motivação, compreensão e paciência ao longo do desenvolvimento deste trabalho.

Ao professor Paulo Rodacki que contribuiu com as ideias iniciais deste trabalho.

Por fim, mas não menos importante, ao professor Dalton Solano dos Reis, orientador deste trabalho, que mesmo diante das adversidades encontradas ao longo desta jornada, sempre se mostrou solícito e interessado em contribuir para o desenvolvimento deste trabalho.

A vida é como andar de bicicleta. Para manter o seu equilíbrio você precisa continuar andando.

Albert Einstein

RESUMO

Este trabalho apresenta a implementação de um aplicativo de visualização de imagens 360 graus interativas para a plataforma Android. A aplicação permite a visualização de imagens a partir das projeções cúbica e esférica, estas projeções são feitas utilizando OpenGL ES 1.0. O usuário pode interagir com a aplicação através da tela multitoque do aparelho, tanto para fazer a navegação na imagem quanto para aplicar *zoom* a imagem selecionada. Para fazer a navegação também é possível utilizar o acelerômetro do dispositivo. Neste trabalho foi possível explorar a plataforma Android e seus recursos, principalmente da biblioteca gráfica nativa. Por fim, este trabalho apresenta alguns testes realizados baseados nos resultados obtidos no desenvolvimento deste aplicativo, utilizando o simulador da plataforma e também um dispositivo móvel.

Palavras-chave: Android. OpenGL. Computação gráfica.

ABSTRACT

This work presents the implementation of an application for image viewing 360 degree interactive for the Android platform. The application allows the viewing of images from the cubic and spherical projections, these projections are made using OpenGL ES 1.0. The user can interact with the application through the multitouch screen of the device, both to make browsing the image and zoom to the selected image. To make the navigation is also possible to use the device the accelerometer. This work was able to explore the Android platform and its features, especially the native graphics library. Finally, this work presents some tests based on the results obtained in developing this application using the simulator platform and also a mobile device.

Key-words: Android. OpenGL ES. Computer graphics.

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de projeção equiretangular 16
Figura 2 – Exemplo de projeção cilíndrica 17
Figura 3 – Exemplo de projeção *fisheye* 18
Figura 4 – Exemplo de projeção cúbica 19
Figura 5 – Diagrama de principais componentes do Android 20
Figura 6 – Construção da esfera no OpenGL ES 22
Figura 7 – Imagem do PTViewer 24
Figura 8 – Tela do Photaf 25
Figura 9 – Produto na ferramenta Web Rotate 360 26
Figura 10 – Diagrama de casos de uso do aplicativo. 28
Quadro 1 – Caso de uso UC01 29
Quadro 2 – Caso de uso UC02 30
Quadro 3 – Caso de uso UC03 31
Quadro 4 – Caso de uso UC04 32
Figura 11 – Diagrama de pacotes 33
Figura 12 – Diagrama de classe do pacote `br.furb.viewer` 34
Figura 13 – Diagrama de classe do pacote `br.furb.viewer.objects` 35
Figura 14 – Diagrama de classe do pacote `br.furb.viewer.renderer` 37
Figura 15 – Diagrama de classe do pacote `br.furb.viewer.util` 39
Figura 16 – Diagrama de sequência do aplicativo 40
Quadro 5 - Método `onSurfaceCreated` 42
Quadro 6 - Método `onSurfaceChanged` 43
Quadro 7 - Método `onDrawFrame` 43
Quadro 8 – Tratamento de toque do `Usuário` 44
Quadro 9 – Tratamento de multitoque do `Usuário` 45
Quadro 10 – Carregar vértices do cubo 46
Quadro 11 – Desenho do cubo 47
Quadro 12 – Carregamento dinâmico das imagens do cubo 48
Quadro 13 – Primeira parte do algoritmo que constrói a esfera 49
Quadro 14 – Segunda parte do algoritmo que constrói a esfera 50

Figura 17 – Tela inicial do aplicativo 51
Figura 18 – Escolha da projeção..... 51
Figura 19 – Escolher modo de interação..... 52
Figura 20 – Escolher imagem para projeção 52
Figura 21 – Rotação no eixo x..... 53
Figura 22 – Rotação no eixo y..... 53
Figura 23 – *Zoom in* no na imagem 54
Figura 24 – Informações do FPS 54
Quadro 15 – Carregamento antigo das texturas do cubo. 55
Quadro 16 – Carregamento atual das texturas do cubo..... 55
Figura 25 – Comparação da taxa de FPS das duas projeções 60
Figura 26 – Comparação de alocação de memória das duas projeções 60
Figura 27 – Exemplo de imagem projetada..... 61
Quadro 17 – Comparação com os trabalhos correlatos..... 61

LISTA DE TABELAS

Tabela 1 - Teste de FPS na projeção esférica..... 56
Tabela 2 – Memória *heap* alocada na projeção esférica 57
Tabela 3 – Teste de FPS na projeção esférica (simulador) 57
Tabela 4 – Memória *heap* alocada na projeção esférica (simulador) 57
Tabela 5 – Teste de FPS na projeção cúbica 58
Tabela 6 – Memória *heap* alocada na projeção cúbica 58
Tabela 7 – Teste de FPS na projeção cúbica (simulador)..... 58
Tabela 8 – Memória *heap* alocada na projeção cúbica (simulador) 59

LISTA DE SIGLAS

API – *Application Programming Interface*

CAD – *Computer-Aided Design*

ADT – *Android Development Tools*

DVM – *Dalvik Virtual Machine*

FPS – *Frames per Second*

GLUT – *Graphics Library Utility Toolkit*

GPU – *Graphics Processing Unit*

HTML – *HyperText Markup Language*

IDE – *Integrated Development Environment*

iOS – *Iphone OS*

OO – Orientação a Objetos

OpenGL ES – *Open Graphics Library for Embedded Systems*

QTVR – *Quick Time Virtual Reality*

SDK – *Software Development Kit*

UML – *Unified Modeling Language*

XML – *eXtensible Markup Language*

SUMÁRIO

1 INTRODUÇÃO 13
1.1 OBJETIVOS DO TRABALHO 14
1.2 ESTRUTURA DO TRABALHO 14
2 FUNDAMENTAÇÃO TEÓRICA 15
2.1 PROJEÇÃO DE IMAGENS 15
2.1.1 Projeção Equiretangular 15
2.1.2 Projeção Cilíndrica 16
2.1.3 Projeção *fisheye* 17
2.1.4 Projeção Cúbica 18
2.2 PLATAFORMA ANDROID 19
2.2.1 Arquitetura Android 20
2.2.2 PanoramaGL 21
2.3 OPENGL ES 21
2.3.1 Desenho dos objetos 22
2.4 TRABALHOS CORRELATOS 23
2.4.1 PTViewer 23
2.4.2 Photoaf 3D 24
2.4.3 Web Rotate 360 25
3 DESENVOLVIMENTO 27
3.1 REQUISITOS PRINCIPAIS DO PROBLEMA A SER TRABALHADO 27
3.2 ESPECIFICAÇÃO 27
3.2.1 Casos de Uso 28
3.2.2 Diagrama de classes 32
3.2.3 Diagrama de sequência 39
3.3 IMPLEMENTAÇÃO 40
3.3.1 Técnicas e ferramentas utilizadas 40
3.3.2 O aplicativo FurbViewer 41
3.3.2.1 A classe `ViewerRenderer` 41
3.3.2.2 A classe `ViewerSurfaceView` 44
3.3.2.3 A classe `Cube` 45
3.3.2.4 Carregar textura 47

3.3.2.5 Desenho da esfera 48
3.3.3 Operacionalidade do aplicativo 50
3.4 RESULTADOS E DISCUSSÃO 55
3.4.1 Projeção Esférica 56
3.4.2 Projeção Cúbica 57
3.4.3 Comparação entre as projeções 59
3.4.4 Comparação com os trabalhos correlatos 61
4 CONCLUSÕES 62
4.1 EXTENSÕES 63
REFERÊNCIAS BIBLIOGRÁFICAS 64

1 INTRODUÇÃO

A demanda por telefones celulares teve uma explosão na última década. Com um mercado aquecido, inovações na área são constantes. Tendo a inovação como principal motivação, o Google em uma parceria com mais de vinte empresas lançou a plataforma Android que consiste em um sistema operacional para *smartphones* e dispositivos do tipo *tablet*, que vem obtendo uma parcela considerável do mercado de dispositivos móveis (PEREIRA; SILVA, 2009, p. 1-10). Com o surgimento dos *smartphones*, já se começa a falar em um processo de substituição de computadores pessoais por *smartphones*, o que leva esta nova demanda a estimular o desenvolvimento de novas aplicações para *smartphones* com funcionalidades parecidas com as dos computadores pessoais. Nesta mesma linha, a computação gráfica também é um mercado em franca expansão, que, segundo Azevedo e Conci (2003, p. 8), se algo puder ser imaginado pelo homem, isso pode ser gerado com a computação gráfica.

A evolução da computação gráfica, em especial das técnicas de processamento de imagens digitais, provocou sensíveis mudanças na forma de observar o mundo através da linguagem visual. Com estes novos recursos disponíveis surgiram várias formas com poucas limitações de visualizar, inventar e transformar uma imagem. Dentre estes novos recursos está a fotografia panorâmica 360 graus que consiste em uma técnica de fotografia que utiliza equipamentos específicos para capturar uma série de imagens que posteriormente são unificadas. Junto com esta técnica existem algoritmos para fazer a união das imagens, assim como permitir a visualização das mesmas (CARVALHO et al., 2009, p. 8).

Diante do exposto, foi criado um aplicativo que une a computação gráfica com a plataforma Android com o propósito de visualizar imagens panorâmicas 360 graus em um dispositivo móvel que roda o sistema operacional Android, proporcionando uma interface simples, com uma boa sensação de imersão ao interagir com o sistema. Para isso foram estudadas as técnicas existentes para projeção de imagens 360 graus, para melhor aplicação destas na construção do aplicativo, em combinação com o uso da *Open Graphics Library for Embedded Systems* (OpenGL ES). Este trabalho também utilizou alguns dos recursos já disponíveis para a plataforma, no intuito de focar o máximo possível no resultado do trabalho.

1.1 OBJETIVOS DO TRABALHO

O objetivo deste trabalho é desenvolver uma aplicação na plataforma Android que possibilite a visualização de imagens 360 graus interativas.

Os objetivos específicos do trabalho são:

a) utilizar como entrada imagens 360 graus já existentes;
b) disponibilizar a visualização das imagens 360 graus de acordo com as técnicas de projeções das imagens estudadas, tais como cúbica e equiretangular (esférica);
c) disponibilizar a projeção das imagens na plataforma Android;
d) disponibilizar as funcionalidades que permitam o usuário fazer a interação com a imagem panorâmica.

1.2 ESTRUTURA DO TRABALHO

Este documento está estruturado em quatro capítulos, sendo que no segundo capítulo é apresentada a fundamentação teórica necessária para o desenvolvimento deste trabalho.

O terceiro capítulo apresenta aspectos do desenvolvimento da aplicação na plataforma Android, os casos de uso, os diagramas de classe e toda a especificação que define a aplicação. Ainda no terceiro capítulo são apresentadas as partes principais da implementação e também os resultados obtidos neste trabalho.

O quarto capítulo refere-se às conclusões do presente trabalho e sugestões para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Na seção 2.1 é apresentada a projeção de imagens e seus tipos. A seção 2.2 trata da plataforma Android e suas características. A seção 2.3 aborda a API OpenGL ES. Por último, a seção 2.4 aborda três trabalhos correlatos.

2.1 PROJEÇÃO DE IMAGENS

A projeção de uma imagem ocorre sempre que uma imagem plana é mapeada sobre uma superfície curva, ou vice-versa, e isso é particularmente comum na fotografia panorâmica. Para ângulos de visão pequena é relativamente fácil distorcer isso em uma imagem em uma folha de papel, uma vez que este arco de visão é relativamente plano. Alguma distorção é inevitável quando se tenta, por exemplo, mapear uma imagem esférica em uma superfície plana, pois cada tipo de projeção apenas tenta minimizar um tipo de distorção. À medida que aumenta o ângulo de visualização, o arco de visualização torna-se mais curvo, e assim a diferença entre os tipos de projeções panorâmicas tornam-se mais evidentes (MCHUGH, 2010). As projeções equiretangulares, cilíndricas, *fisheye* e cúbica são apresentadas a seguir.

2.1.1 Projeção Equiretangular

Uma projeção equiretangular ou esférica é uma projeção que mapeia a latitude e longitude de um globo esférico diretamente em coordenadas horizontais e verticais em um *grid*, onde o *grid* tem uma relação 2:1 de largura e altura. A imagem vertical permanece vertical, já o eixo horizontal tende a ficar mais esticado à medida que se aproxima dos pólos (MCHUGH, 2010). Sua maior aplicação, devido a sua característica de igual espaçamento entre os paralelos e meridianos, é o tratamento como um sistema de coordenadas plano cartesiano, sendo portanto, de fácil manipulação em sistemas de *Computer-Aided Design* (CAD) tais como AutoCad (AUTODESK, 2011), MicroStation (BENTLEY, 2011) entre

outros, que não comportam sistemas de projeção cartográficos. Projeções equiretangulares podem mostrar todo o ângulo horizontal de até 360 graus e o vertical de 180 graus (VALENTE, 2010, p. 9). Este tipo de projeção possui uma quantidade muito grande de dados redundantes perto dos polos, pelo fato da imagem ficar muito esticada, quando for necessário reduzir a resolução de uma imagem, como consequência os pólos não serão reduzidos Esta projeção é utilizada em um dos trabalhos correlatos, o PTViewer (PTVIEWER, 2006).

A Figura 1 apresenta um exemplo de projeção equiretangular.

Fonte: Panotools (2008b).

Figura 1 – Exemplo de projeção equiretangular

2.1.2 Projeção Cilíndrica

Projeções cilíndricas são semelhantes às projeções equiretangulares, exceto que os objetos das imagens também ficam esticados verticalmente, à medida que se aproximam dos polos norte e sul da imagem. Desta forma, os objetos tendem a esticarem infinitamente nos polos, portanto nenhuma linha horizontal é mostrada na parte inferior e superior do *grid* achatado. É por esta razão que as projeções cilíndricas não são muito adequadas para as imagens com ângulos muito grandes de visão vertical. Projeção cilíndrica é também o tipo padrão utilizado por tradicionais câmeras de filmes panorâmicos com lentes do tipo *swing* (MCHUGH, 2010).

A Figura 2 apresenta um exemplo de projeção cilíndrica.

Fonte: Panotools (2008a).

Figura 2 – Exemplo de projeção cilíndrica

2.1.3 Projeção *fisheye*

Projeções do tipo *fisheye* são aquelas que representam uma porção significativa do ambiente em um plano, ao abrir mão da conformidade de representação das linhas retas. Como porção significativa entende-se um campo de visão que cobre mais do que 120 graus (RAY, 2002, p. 331). Esta projeção tem como objetivo criar um *grid* achatado onde a distância do centro deste *grid* é aproximadamente proporcional ao ângulo de visão real, produzindo uma imagem que seria semelhante ao reflexo de uma esfera metálica. Essa geralmente não é utilizada como um formato de saída para fotografias panorâmicas, mas pode em alternativa representar as imagens de entrada quando o tipo da lente da câmera sendo utilizada para captura da foto é uma lente grande angular do tipo *fisheye* (MCHUGH, 2010). Além disso, este tipo de projeção também é útil porque ao fazer um panorama imersivo utiliza-se menos fotos do que as outras projeções (VALENTE, 2010, p. 27). A Figura 3 apresenta um exemplo de projeção *fisheye*.

Fonte: Panotools (2008c).

Figura 3 – Exemplo de projeção *fisheye*

2.1.4 Projeção Cúbica

Projeção cúbica é um tipo de projeção que divide a imagem nas seis faces de um cubo. A divisão é feita da seguinte forma: frente, trás, direita, esquerda, zênite e nadir, onde cada um com 90 graus por 90 graus de visualização. As imagens são costuradas de forma que suas arestas não visíveis fiquem unidas. Entretanto, se a imagem for muito ampliada, as arestas do cubo podem começar a aparecer, sendo esta uma desvantagem para imagens como muita costura (JACOBS, 2004, p. 7).

A Figura 4 apresenta um exemplo de projeção cúbica.

Fonte: Panotools (2007).

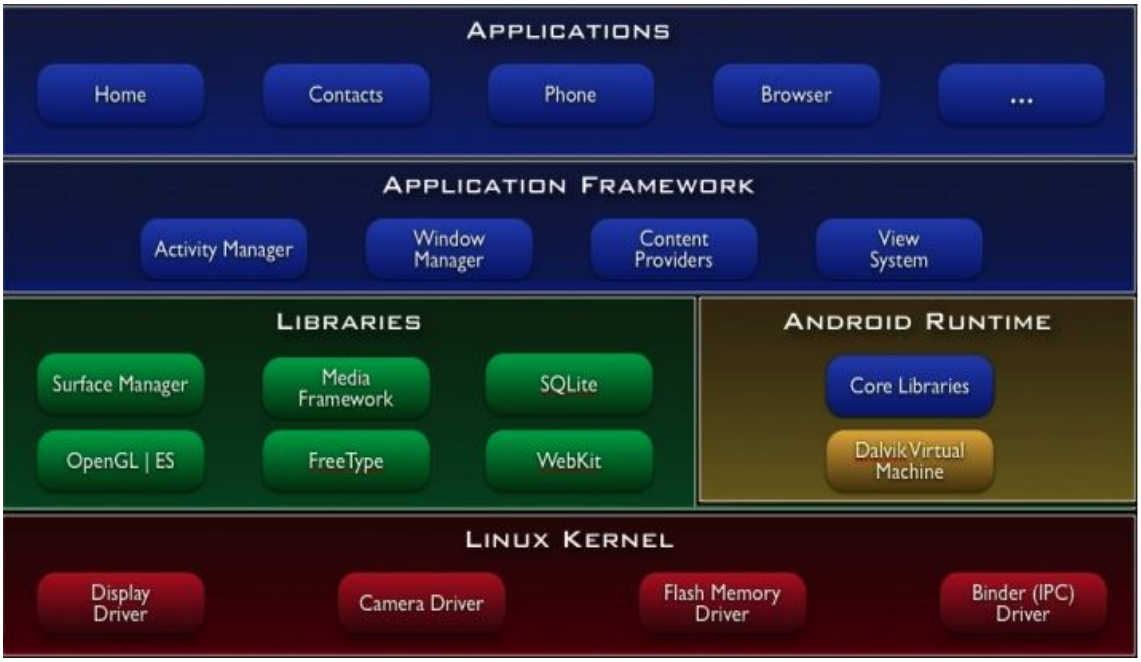
Figura 4 – Exemplo de projeção cúbica

2.2 PLATAFORMA ANDROID

Segundo o *site* oficial (GOOGLE, 2011a), Android é uma pilha de *softwares* que inclui um sistema operacional com um *kernel* Linux, *middleware* e as aplicações essenciais para um dispositivo móvel. A plataforma oferece novas possibilidades para aplicações de dispositivos móveis uma vez que oferece um ambiente de desenvolvimento aberto que utiliza um *kernel* Linux *open-source*. O acesso ao hardware está disponível para todas as aplicações através de várias bibliotecas. No Android todas as aplicações têm a mesma prioridade. Aplicações de terceiros e aplicações nativas são escritas utilizando as mesmas bibliotecas (MEIER, 2010, p. 2). É importante destacar ainda que as aplicações são executadas em uma máquina virtual denominada de *Dalvik Virtual Machine* (DVM), que é uma máquina virtual otimizada para dispositivos móveis.

2.2.1 Arquitetura Android

A Figura 5 apresenta as camadas da plataforma Android.



Fonte: Google (2011a).

Figura 5 – Diagrama de principais componentes do Android

Segundo Google (2011a), a arquitetura é dividida da seguinte forma:

a) Linux *kernel*: responsável pelos serviços centrais do sistema, além da segurança, gerenciamentos de memória, processos, rede e *drivers*. Atua também como uma camada de abstração entre o hardware e o *software*;
b) Android *runtime*: inclui um conjunto de bibliotecas que fornece a maioria das funcionalidades disponíveis nas principais bibliotecas da linguagem Java;
c) *libraries*: carrega um conjunto de bibliotecas C/C++ utilizadas por vários componentes do sistema. Está incluída neste conjunto a biblioteca C padrão (`Libc`), além de funções para navegadores web, de aceleração de hardware, renderização 3D e funções de acesso a biblioteca SQLite;
d) *application framework*: permite ao desenvolvedor construir aplicações extremamente ricas e inovadoras, através de acesso livre ao hardware, acesso às informações locais, execução de serviços em *background*, etc.;
e) *applications*: encontram-se todos os aplicativos do Android, como cliente de *e-mail*, contatos, navegador *web* e as aplicações de desenvolvedores.

2.2.2 PanoramaGL

PanoramaGL é uma biblioteca para visualizar imagens panorâmicas no Android. Esta biblioteca foi originalmente desenvolvida para o *Iphone OS* (IOS), porém foi portada para o Android. Esta biblioteca foi desenvolvida especificamente para desenvolver aplicativos no Android relacionados com imagens panorâmicas. Esta biblioteca contém elementos para criação da imagem panorâmica, assim como a sua visualização no Android. Algumas características da biblioteca são (BAEZ; ÑUÑAY, 2010):

a) suporta OpenGL ES 1.0;
b) testada na versões 2.0 a 2.2 do Android SDK;
c) suporta projeções cúbicas, esféricas e cilíndricas;
d) suporta o uso do acelerômetro do dispositivo para interagir com as imagens;
e) suporta funções de *zoom*.

2.3 OPENGL ES

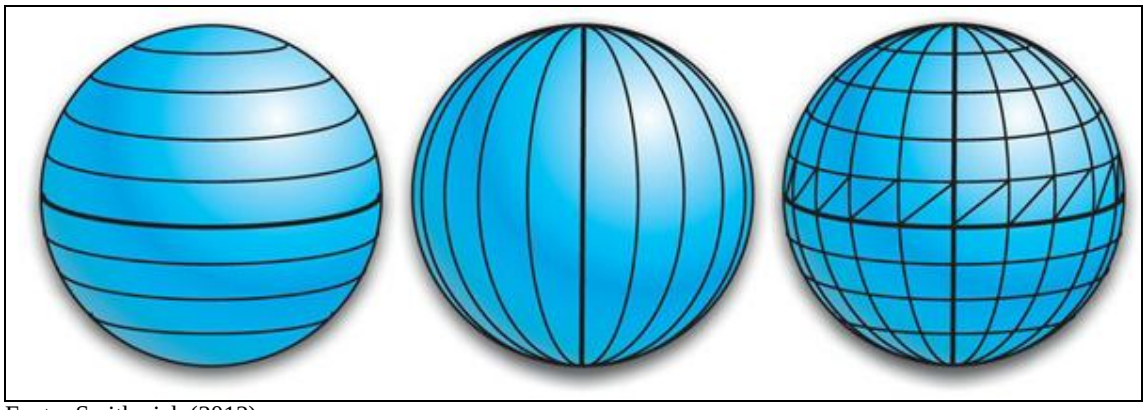
OpenGL ES é uma versão compacta da biblioteca gráfica OpenGL. É uma API de renderização em baixo nível adaptada para sistemas embarcados. A primeira versão da OpenGL ES tinha como objetivo prover uma API extremamente compacta sem diminuir seus recursos. Teve que ser implementada totalmente por *software* e com tamanho menor do que 50kb e ao mesmo tempo ser bem adequada à aceleração de *hardware*, assim como tornar os familiares gráficos dos computadores pessoais disponíveis em sistemas embarcados. Com a evolução das versões, nos dias atuais é utilizada desde relógios de pulsos a modernos *smartphones* e nos mais poderosos console atuais (PULLI et al., 2008, p. 37-40).

A plataforma Android inclui suporte para gráficos 2D e 3D de alto desempenho, utilizando OpenGL ES. A plataforma suporta a versão OpenGL ES 1.0 desde a versão 1.0 do Android. A partir da versão 2.2 o Android passou a suportar OpenGL ES 2.0 (GOOGLE, 2011b). Entretanto é importante ressaltar que com a OpenGL ES 2.0 não é possível utilizar o emulador da plataforma para fazer os testes. A única forma de testar as aplicações desenvolvidas na plataforma é utilizando algum dispositivo que suporta Android (GOOGLE, 2011b). Apesar deste empecilho para os desenvolvedores, a OpenGL ES pode ser muito

poderosa, pois o desenvolvedor pode escrever código diretamente para a *Graphics Processing Unit* (GPU), que são compilados e interpretados em tempo real. A API também permite fazer chamadas utilizando código C, o que pode tornar o processamento ainda mais rápido (KOMATINENI; MACLEAN; HASHIMI, 2011, p. 633).

2.3.1 Desenho dos objetos

Embora o OpenGL seja uma poderosa plataforma para desenvolvimento baixo nível, ainda deixa muito a desejar quando se trata de alguma coisa de alto nível. Por isso, quando o assunto é modelagem de objetos, várias bibliotecas externas podem fazer o serviço. Porém dependendo do caso, a performance da aplicação pode ser afetada. A versão padrão do OpenGL possui um *toolkit* muito popular conhecido como *Graphics Library Utility Toolkit* (GLUT), que permite o desenvolvedor utilizar conceitos básicos de interface do usuário, gerenciamento de telas e desenho de alguns objetos básicos, incluindo esferas. Entretanto, até o presente momento, não existe nenhuma versão oficial do GLUT para Android, por isso, algumas formas simples como a esfera, podem ser desenhas utilizando somente geometria básica e utilizando somente a plataforma Android, melhorando a performance da aplicação (SMITHWICK; MAYANK, 2012, p. 65). A Figura 6 apresenta o modo como é construída a esfera utilizando o conceito do OpenGL ES para o Android.



Fonte: Smithwick (2012).

Figura 6 – Construção da esfera no OpenGL ES

2.4 TRABALHOS CORRELATOS

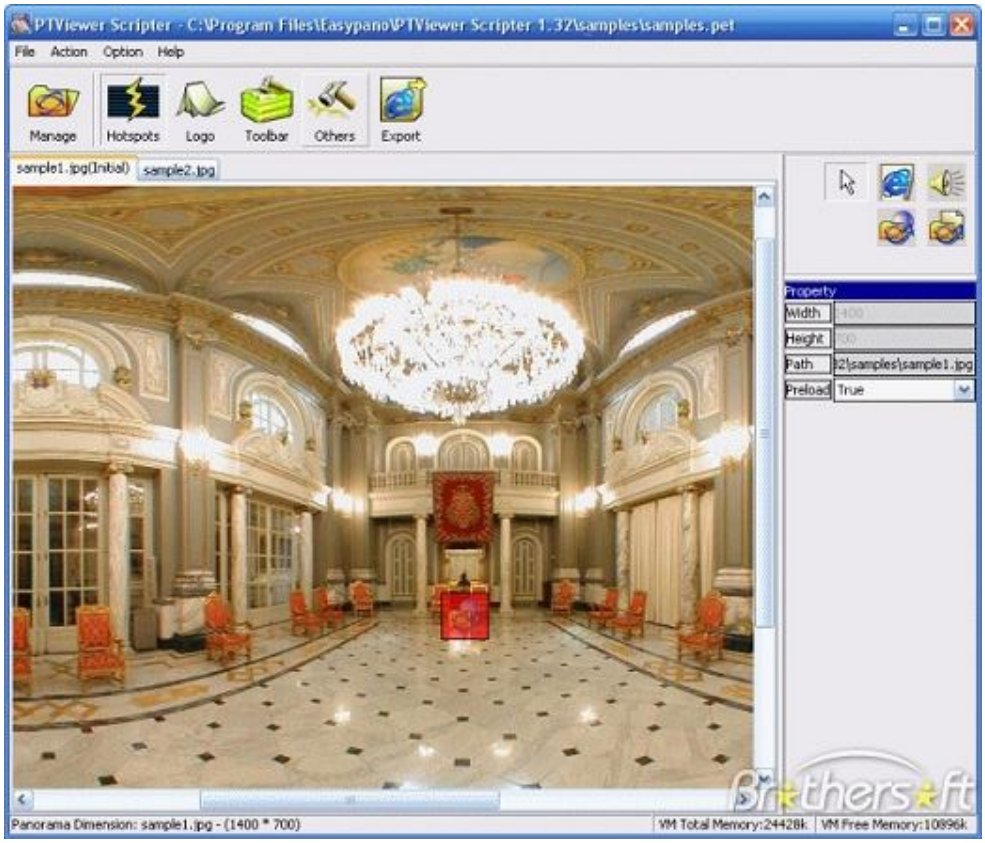
Aqui são apresentados trabalhos correlatos para computadores pessoais e plataforma Android, que permitem visualizar imagens em 360 graus, os quais são: PTViewer (PTVIEWER, 2006), Photoaf 3D (ANDROID MARKET, 2011) e Web Rotate 360 (WEB ROTATE 360 LLC, 2011).

2.4.1 PTViewer

PTViewer é uma Java *applet* desenvolvida para visualizar imagens panorâmicas 360 graus. A ferramenta foi desenvolvida por Helmut Dersch da Universidade de Furtwangen na Alemanha. Como o código fonte é *open-source*, existem várias versões do PTViewer. Sua principal característica é a utilização de imagens com projeções cilíndricas e equiretangulares (PTVIEWER, 2006). Outras características do PTViewer, conforme Dersch (2001), são:

a) gravação de panoramas esféricos e cilíndricos, com um campo de visão horizontal de até 360 graus e campo de visão vertical até 180 graus;
b) gravação de panoramas no mesmo formato da ferramenta Quick Time Virtual Reality (QTVR) (APPLE, 2011);
c) visualização de imagens retilíneas usando a extensão opcional PTZoom (PTVIEWER, 2006);
d) renderização de alta qualidade como encontrado nos melhores *plug-ins* de código fechado;
e) tamanho da imagem e da janela limitados somente pelo tamanho da memória;
f) é possível usar *script* via *Hypertext Markup Language* (HTML), Javascript e um sistema interno de *script*;
g) proteção adicional de imagens e panoramas através de encriptação.

A Figura 7 apresenta uma imagem do PTViewer.



Fonte: Dersch (2001).

Figura 7 – Imagem do PTViewer

2.4.2 Photoaf 3D

Photoaf 3D é um aplicativo desenvolvido exclusivamente para a plataforma Android, o que permite que o usuário utilize a câmera para capturar imagens para formar um panorama e posteriormente visualizá-los. O aplicativo é distribuído gratuitamente na Android Market, porém, possui uma versão paga. Este aplicativo foca na captura de imagens, não em sua visualização, entretanto, é um dos poucos disponíveis para a plataforma. Alguns recursos do aplicativo são (ANDROID MARKET, 2011):

a) usar a interface para dar *zoom* nas imagens;
b) fazer a costura automática das imagens;
c) transferir o panorama criado para um computador pessoal;
d) otimizado para a versão 2.3 do Android;
e) funciona em *tablets* que possuem a plataforma Android.

A Figura 8 mostra uma tela do Photoaf 3D.

Fonte: Android Market (2011).

Figura 8 – Tela do Photaf

2.4.3 Web Rotate 360

Web Rotate 360 Viewer é uma ferramenta simples e poderosa de uso estritamente comercial, para publicidade de produtos *on-line*. A ferramenta não requer tecnologia do lado do servidor, somente são necessários alguns arquivos de Javascript e Flash que executem na página. Quando uma página é carregada, o sistema automaticamente baixa as imagens configuradas do *web server* de acordo com as configurações do cliente. O usuário não necessariamente precisar ter conhecimento técnico de linguagens de programação, a configuração é feita toda através de um arquivo *eXtensible Markup Language* (XML). Para fazer a junção das imagens o sistema utiliza as imagens fornecidas pelo cliente. Alguns recursos da ferramenta são (WEB ROTATE 360 LLC, 2011):

a) funciona em iPad, iPhone e em outros dispositivos que contenham *browsers*, mesmo sem Flash;
b) toda configuração é feita em um arquivo XML;
c) o sistema é altamente configurável (tamanho, recursos, cores, etc.);
d) contém um software grátis para uma configuração rápida.

A Figura 9 mostra um produto projetado na ferramenta Web Rotate 360.

Fonte: Web Rotate 360 LLC (2011).

Figura 9 – Produto na ferramenta Web Rotate 360

3 DESENVOLVIMENTO

Neste capítulo são detalhadas as etapas do desenvolvimento do aplicativo. Também pode ser encontrado em Iten Jr. (2012) uma descrição da documentação dos fontes. São apresentados os requisitos principais, a especificação, a implementação e ao final são apresentados os resultados e discussão.

3.1 REQUISITOS PRINCIPAIS DO PROBLEMA A SER TRABALHADO

O aplicativo deverá:

a) permitir visualizar um panorama de 360 graus já formado, na plataforma Android (Requisito Funcional - RF);
b) permitir pelo menos dois formatos de imagens panorâmicas como entrada (RF);
c) deverá ter funções básicas de navegação (direita, esquerda, cima, baixo), assim como função de *zoom in* e *zoom out* (RF);
d) utilizar o multitoque para interagir com as funções do aplicativo (RF);
e) ser desenvolvido na linguagem Java (Requisito Não Funcional - RNF);
f) utilizar o Eclipse IDE como ambiente de desenvolvimento, juntamente com o Android SDK (RNF);
g) ser compatível com o Android 2.2 ou superior (RNF);
h) ser compatível com pelo menos um dispositivo móvel que suporte a plataforma Android (RNF).

3.2 ESPECIFICAÇÃO

A especificação deste trabalho foi desenvolvida através da ferramenta Enterprise Architect utilizando conceitos de Orientação a Objetos (OO) através dos diagramas de caso de uso, classe e sequência da *Unified Modeling Language* (UML).

3.2.1 Casos de Uso

Esta seção descreve todos os casos de uso do aplicativo. Foi identificado apenas o ator `Usuário`, ele faz uso dos recursos do aplicativo para visualizar os panoramas no sistema. Na Figura 10 é apresentado o diagrama de casos de uso do aplicativo.

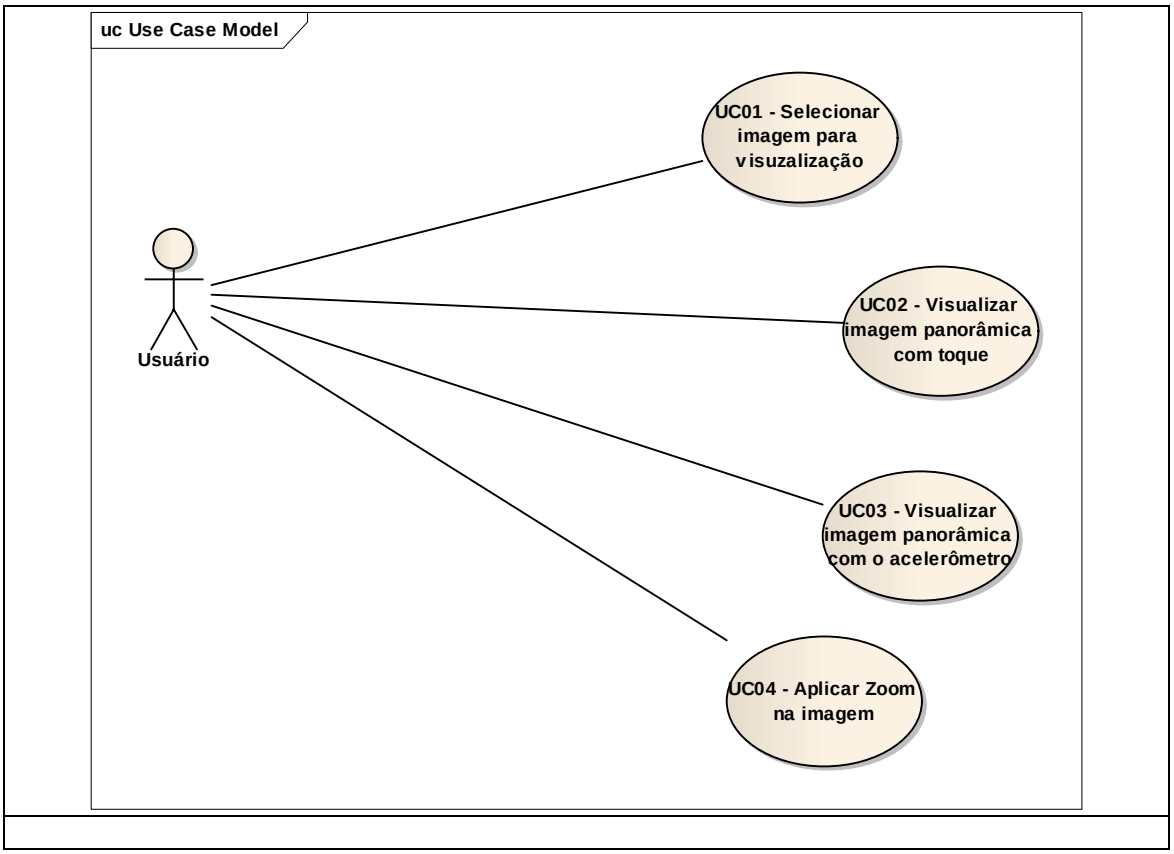


Figura 10 – Diagrama de casos de uso do aplicativo.

3.2.1.1 Selecionar uma imagem para visualização

Este caso de uso apresenta os passos iniciais necessários para o `Usuário` visualizar uma imagem panorâmica, os detalhes deste caso de uso estão descritos no Quadro 1.

UC01 – Selecionar imagem para visualização	
Descrição	O sistema deve permitir que o `Usuário` selecione uma imagem entre as disponíveis, escolha o modo de projeção, o modo de interação e visualize a imagem como um panorama. Deve permitir também que o `Usuário` possa interagir com o sistema.
Pré-Condição	Não há.
Cenário Principal	1. O `Usuário` entra no aplicativo de visualização de panoramas; 2. O sistema exibe a tela inicial do aplicativo com as informações do mesmo; 3. O `Usuário` clica na opção entrar; 4. O sistema exibe uma tela com as miniaturas das imagens disponíveis; 5. O `Usuário` seleciona o tipo de projeção desejado entre os disponíveis (cúbica e esférica); 6. Se o `Usuário` selecionar o tipo de interação `Toque`, ir para o cenário principal do caso de uso UC02; 7. Se o `Usuário` selecionar o tipo de interação `Acelerômetro`, ir para o cenário principal do caso de uso UC03.
Cenário Alternativo 1	1. Se no passo 5 do cenário principal o `Usuário` clicar em voltar o sistema volta para o passo 4.
Cenário Alternativo 2	1. Se no passo 6 do cenário principal o `Usuário` clicar em voltar o sistema volta para o passo 4.
Cenário Alternativo 3	1. Se no passo 7 do cenário principal o `Usuário` clicar em voltar o sistema volta para o passo 4.
Pós-Condição	O sistema deve apresentar a tela com a imagem panorâmica pronta para interagir com o `Usuário`.

Quadro 1 – Caso de uso UC01

3.2.1.2 Visualizar imagem panorâmica através de toque

Este caso de uso relata como é feita a interação com `Usuário` na visualização da

imagem panorâmica 360 graus, utilizando o recurso de multitoque do aparelho. O Quadro 2 fornece maiores detalhes sobre este caso de uso.

UC02 – Visualizar imagem panorâmica com toque	
Descrição	O sistema apresenta a imagem panorâmica solicitada pelo `Usuário`, na projeção desejada. Permite ao `Usuário` opções de navegação pela imagem, tais como girar para direita, esquerda, frente, trás.
Pré-Condição	É necessário que o dispositivo possua recurso de toque.
Cenário Principal	1. O sistema apresenta a imagem panorâmica projetada; 2. Caso o `Usuário` deslize o dedo sobre a tela para direita vá para o passo 6; 3. Caso o `Usuário` deslize o dedo sobre a tela para esquerda vá para o passo 6; 4. Caso o `Usuário` deslize o dedo sobre a tela para cima vá para o passo 6; 5. Caso o `Usuário` deslize o dedo sobre a tela para baixo vá para o passo 6; 6. O sistema pegas as novas coordenadas adquiridas pelo toque do `Usuário` e rotaciona a imagem conforme essas coordenadas.
Cenário Alternativo 1	1. Caso o `Usuário` clique em voltar ao final de qualquer passo, o sistema volta para o passo 5 do cenário Principal do caso de uso UC01.
Cenário Alternativo 2	1. Se o `Usuário` clicar no *label* `info` no canto superior esquerdo da tela o sistema deve mostrar as informações do FPS.
Pós-Condição	Imagem rotacionada conforme toque do `Usuário`.

Quadro 2 – Caso de uso UC02

3.2.1.3 Visualizar imagem panorâmica através do acelerômetro

Este caso de uso relata como é realizada a interação com `Usuário` na visualização da imagem panorâmica 360 graus, utilizando o acelerômetro do aparelho. O Quadro 3 fornece maiores detalhes sobre este caso de uso.

UC03 – Visualizar imagem panorâmica com o acelerômetro	
Descrição	O sistema apresenta a imagem panorâmica solicitada pelo `Usuário`, na projeção desejada. Permite ao `Usuário` opções de navegação pela imagem, através da movimentação do aparelho, tais como girar para direita, esquerda, frente, trás.
Pré-Condição	É necessário que o dispositivo possua acelerômetro.
Cenário Principal	1. O sistema apresenta a imagem panorâmica projetada; 2. Caso o `Usuário` incline o dispositivo para direita vá para o passo 6; 3. Caso o `Usuário` incline o dispositivo para esquerda vá para o passo 6; 4. Caso o `Usuário` incline o dispositivo para cima vá para o passo 6; 5. Caso o `Usuário` incline o dispositivo para baixo vá para o passo 6; 6. O sistema pega as novas coordenadas adquiridas através do acelerômetro e rotaciona a imagem conforme essas coordenadas.
Cenário Alternativo 1	1. Caso o `Usuário` clique em voltar ao final de qualquer passo, o sistema volta para o passo 5 do cenário Principal do caso de uso UC01.
Cenário Alternativo 2	1. Se o `Usuário` clicar no *label* `info` no canto superior esquerdo da tela o sistema deve mostrar as informações do FPS.
Pós-Condição	Imagem rotacionada conforme movimentação do dispositivo.

Quadro 3 – Caso de uso UC03

3.2.1.4 Aplicar *zoom* na imagem

Este caso de uso relata como é feita a interação com o `Usuário` para que seja possível realizar os efeitos de *zoom in* e *zoom out* na visualização da imagem panorâmica de 360 graus. O Quadro 4 fornece maiores detalhes sobre este caso de uso.

UC04 – Aplicar zoom na imagem	
Descrição	Utilizar o recurso de multitoque do aparelho para realizar as funções de *zoom in* e *zoom out*.
Pré-Condição	É necessário que o dispositivo possua o recurso multitoque.
Cenário Principal	1. Caso o `Usuário` pressione a tela com dois dedos realizando o gesto de aproximação dos dedos vá para o passo 4; 2. Caso o `Usuário` pressione a tela com dois dedos realizando o gesto dos dedos se afastando vá para o passo 3; 3. O sistema aplica o *zoom in* na imagem e mostra a imagem novamente; 4. O sistema aplica o *zoom out* na imagem e mostra a imagem novamente.
Cenário Alternativo 1	2. Caso o `Usuário` clique em voltar ao final de qualquer passo, o sistema volta para o passo 5 do cenário Principal do caso de uso UC01.
Pós-Condição	A imagem deverá estar com o *zoom* aplicado.

Quadro 4 – Caso de uso UC04

3.2.2 Diagrama de classes

A seguir são apresentadas as classes necessárias para o desenvolvimento do aplicativo. Para uma melhor visualização do agrupamento de classes a Figura 11 apresenta um diagrama de pacotes, suas dependências bem como as classes que o compõem.

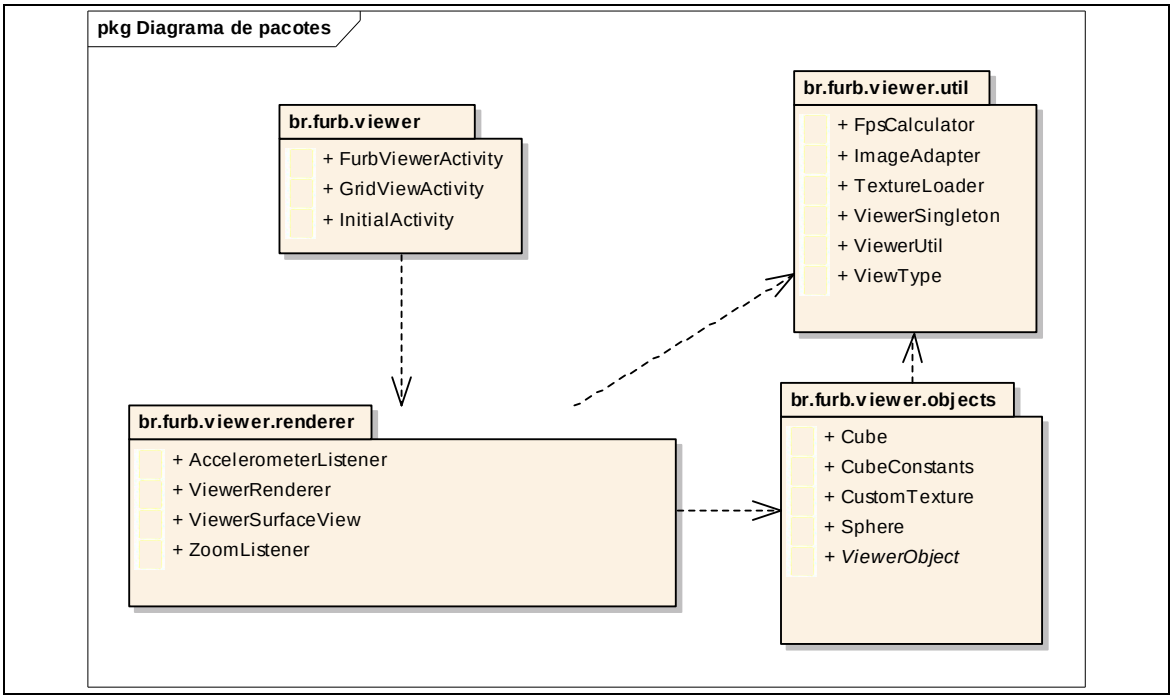


Figura 11 – Diagrama de pacotes

3.2.2.1 Pacote `br.furb.viewer`

O pacote `br.furb.viewer` (Figura 12) é o principal pacote da aplicação. Nele estão contidas as principais classes do sistema. Por padrão, as classes deste pacote herdam as funcionalidades da classe `Activity` da API do Android, que gerencia o ciclo de vida da aplicação.

A classe `InitialActivity` apresenta apenas a tela inicial para o `Usuário`, é classe que inicia o fluxo do sistema.

A classe `GridViewActivity` é a classe responsável por apresentar ao `Usuário` as imagens disponíveis para visualização panorâmica. Este classe também recebe do `Usuário` as informações sobre o modo de projeção e tipo de interação. Após a escolha do `Usuário` essa classe chama a classe `FurbViewerActivity` através do mecanismo de intenções, passando como parâmetro a imagem escolhida para visualização, o modo de projeção e o tipo de interação.

A classe `FurbViewerActivity` é a classe que efetivamente inicia as atividades referentes a visualização da imagem. Possui uma atributo do tipo `GLSurfaceView` que no método `onCreate` da classe é colocado como `View` padrão da aplicação, ainda neste método

recebe o índice da imagem escolhida para visualização e chama o método `loadFaces` da classe `ViewerSingleton` para carregar as imagens necessárias para a projeção. Assim como, prepara a classe `ViewerSurfaceView` conforme escolha de interação do `Usuário` (toque o Acelerômetro). Definindo que caso o modo de interação seja o acelerômetro a orientação do dispositivo será "retrato".

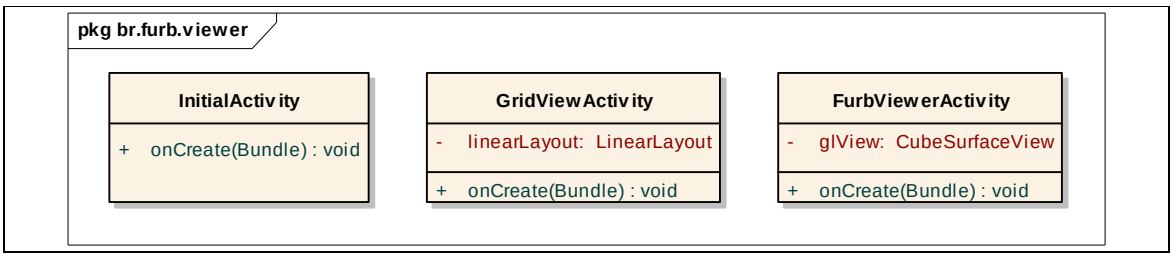


Figura 12 – Diagrama de classe do pacote `br.furb.viewer`

3.2.2.2 Pacote `br.furb.viewer.objects`

O pacote `br.furb.viewer.objects` (Figura 13) é o pacote que contém as classes necessárias para realizar a projeção das imagens.

A classe `ViewerObject` é a classe padrão para todos os objetos visualizados na tela, cada objeto que será mostrado na tela, herda desta classe. Ela possui um método abstrato chamado `draw` que deverá ser implementado nas classes que herdam desta. Pois este método é chamado pelo renderizador no momento em que os objetos são mostrados na tela. Possui como atributo um mapa do tipo `HashMap` que armazena as referências para as texturas necessárias para realizar a projeção, um `Buffer` que armazena as coordenadas do objeto, e ainda outro `Buffer` com as coordenadas das texturas. O método `enableVertex` é responsável por iniciar o `Buffer` de vértices e habilitá-lo para o uso pelo OpenGL. O método `enableTextures` tem a mesma funcionalidade do anterior, porém para o `Buffer` de texturas. O método `enableGLFunctions` ativa funções do OpenGL que são necessárias para as projeções, o método `disableGLFunctions` desativa estas funções.

A classe `Cube` é uma classe que herda as funcionalidades de `ViewerObject`, pois é um objeto que será mostrado na tela. Esta classe representa um objeto utilizado na projeção cúbica. Por fim, o método `draw` que é herdado de `ViewerObject` faz a projeção do cubo, de modo que cada face do cube é desenhada por vez, utilizando a função `GL_TRIANGLE_STRIP` do OpenGL.

A classe `Sphere` é uma classe que herda as funcionalidades de `ViewerObject`, representa uma esfera utilizada para fazer a projeção esférica. Esta classe é similar a classe `Cube`, porém, tem a implementação do método `draw` apropriado para esfera e possui o método `buildSphere` que efetivamente cria os vértices e os índices para as texturas da esfera.

A classe `CustomTexture` é a classe que representa as texturas a serem projetadas. Possui como atributo um identificador que diferencia as texturas utilizadas, uma referência para a imagem da textura no disco do aparelho. E também possui uma imagem do tipo `Bitmap` que é criado através da classe nativa do Android `BitmapFactory`, no momento em que a classe é instanciada.

A classe `CubeConstants` contém dois arrays do tipo `float`, um `cubeCoords` que representa as coordenadas dos vértices do cubo em x, y, z, outro `cubeTexCoords` que armazena as coordenadas das texturas do cubo, representadas em x, y.

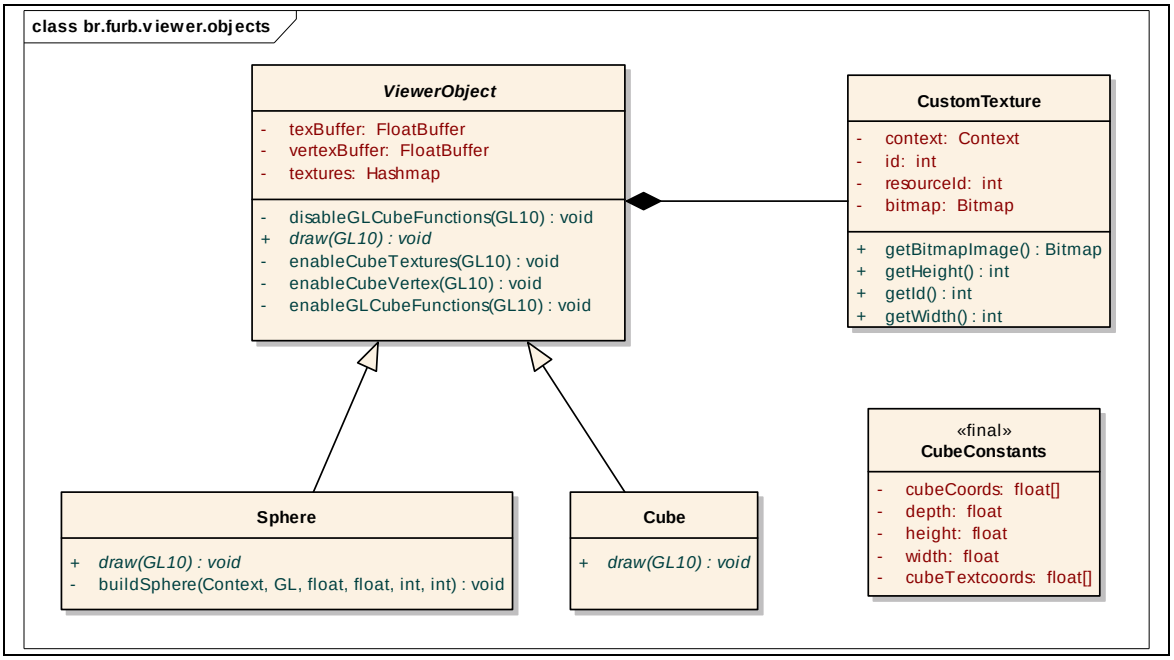


Figura 13 – Diagrama de classe do pacote `br.furb.viewer.objects`

3.2.2.3 Pacote `br.furb.viewer.renderer`

Este pacote (Figura 14) é responsável pela renderização dos objetos e enviá-los para serem mostrados na tela.

A classe `ViewerSurfaceView` é uma extensão da classe `GLSurfaceView` que é uma classe nativa do Android, responsável por criar a tela que o `Usuário` visualiza. A classe

sobrescreve o método `onTouchEvent` que tem como objetivo tratar os eventos de toque na tela do dispositivo. Este método recebe como parâmetro um objeto do tipo `MotionEvent` também nativo do Android. Na implementação da classe `ViewerSurfaceView`, caso o modo de interação seja por toque este método recebe as coordenadas (x, y) do toque na tela. Computa a diferença do ponto atual com o ponto armazenado e passa para o Renderizador os novos valores. Esta classe tem como atributos os valores de x e y a cada evento de toque, possui também uma constante com um valor a ser utilizado na hora da passagem dos valores para o Renderizador. Caso o modo de interação escolhido seja o acelerômetro, no construtor desta classe é criado uma instância da classe `AccelerometerListener`, que passar a escutar as variações no acelerômetro do dispositivo.

A classe `ViewerRenderer` é uma implementação da interface `GLSurfaceView.Renderer` que é uma classe nativa do Android. Esta classe é responsável por renderizar o objeto e mostrá-lo na tela. Os métodos `onSurfaceCreated` e `onSurfaceChanged` são sobrescritos nesta classe. O primeiro trata-se de um *callback* quando a `ViewerSurfaceView` é criada, neste caso alguns parâmetros necessários para visualizar o panorama são passados para o OpenGL tais como `GL_TEXTURE_2D`, `GL_SMOOTH`, entre outros. Já o segundo é chamado sempre que o tamanho da tela muda. Implementa ainda o método `onDrawFrame`, que efetivamente desenha o objeto (cubo ou esfera) chamando o método `draw` de `ViewerObject` e posiciona a câmera do OpenGL no centro do cubo utilizando `gluPerspective`.

A classe `ZoomListener` estende a classe `ScaleGestureDetector.SimpleOnScaleGestureListener` nativa do Android, que é uma classe especifica para tratar eventos do tipo multitoque neste caso, é responsável por tratar os eventos do tipo *zoom*.

A classe `AccelerometerListener` implementa a interface `SensorEventListener` nativa do Android. Esta classe trata os eventos provenientes do acelerômetro, recebe as coordenadas do acelerômetro do aparelho e as repassa para o `ViewerRenderer` que rotaciona a imagem conforme movimentação do aparelho. Essa classe possui implementação de dois métodos `onAccuracyChanged` e `onSensorChanged`, porém, somente este último é utilizado, primeiramente verifica se o sensor que teve mudança é o acelerômetro. Depois, caso seja, passa os valores para o `ViewerRenderer`.

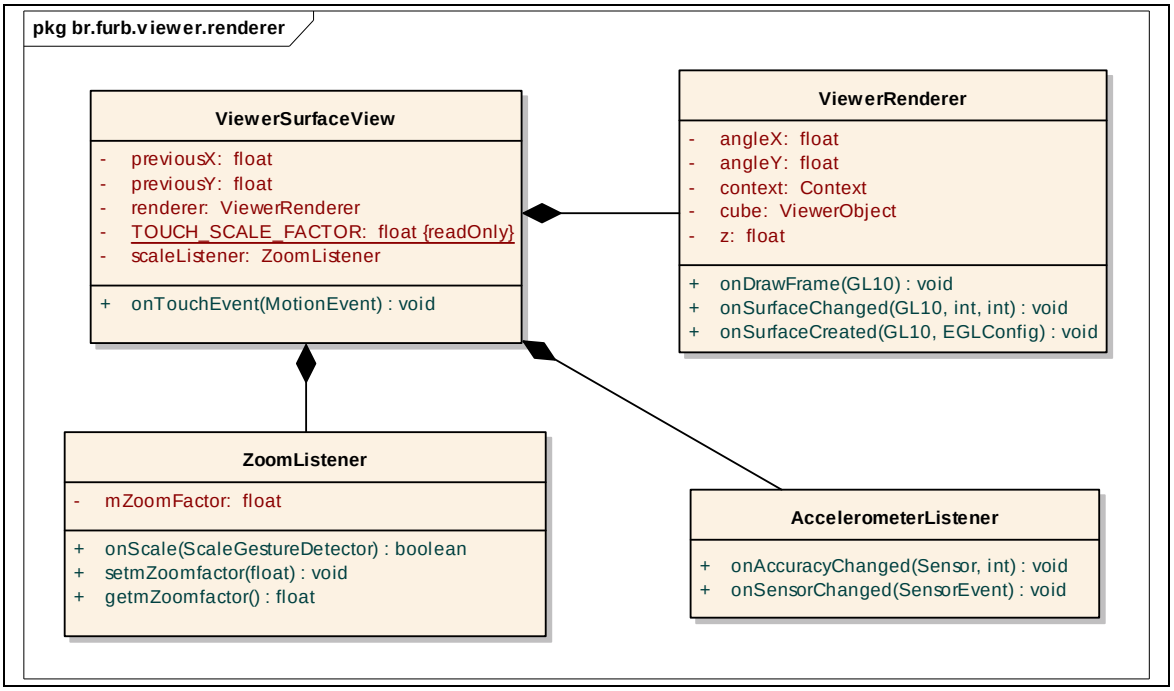


Figura 14 – Diagrama de classe do pacote `br.furb.viewer.renderer`

3.2.2.4 Pacote `br.furb.viewer.util`

O pacote `br.furb.viewer.util` (Figura 15) é um pacote com classes utilitárias usadas em diferentes pontos do aplicativo.

A classe `ViewerUtil` possui dois métodos estáticos, `makeFloatBuffer` que recebe como parâmetro um *array* do tipo `float` que é alocado primeiramente como um `ByterBuffer` que é ordenado pela a ordem nativa da classe padrão do Android `ByteOrder`. Em seguida é convertido para `FloatBuffer` e retornado no método. Este método é utilizado para converter os *arrays* de vértices e texturas para `FloatBuffer` que é o formato esperado pelo OpenGL no método `glVertexPointer`. O segundo método `makeByteBuffer` que recebe como parâmetro uma imagem do tipo `Bitmap`, que é alocado em um `ByteOrder`, em seguida é tratado como um `IntBuffer`, que converte a imagem ARGB para RGBA e por fim, retorna o `ByteBuffer`. Este método é necessário para carregar a textura, pois para o método do OpenGL `glTexImage2D` é necessário que a textura esteja em um `ByteBuffer`.

A classe `TextureLoader` é utilizada para carregar a textura das faces do cubo e a textura da esfera. Contém um método público estático (`loadTexutre`) que recebe como parâmetro um mapa que contém cada face de cubo ou uma imagem esférica única e armazena

um Bitmap de cada imagem que será carregada. Em seguida chama o segundo método loadTexture que recebe como parâmetro o tipo de textura, que neste caso é fixo (GL_TEXTURE_2D) e uma instância da classe CustomTexture. Dentro deste é gerado uma textura utilizando glGenTextures e então é chamado o terceiro e último método loadTexture que recebe o Id da textura gerado no método anterior, as dimensões da imagem e um ByteBuffer da imagem. Neste último método é feito o *bind* da textura para ser utilizada no instante em que o objeto será mostrado. Neste método são passados parâmetros importantíssimos para criar um aspecto de panorama na imagem, os principais parâmetros passados para o OpenGL são GL_CLAMP_TO_EDGE de GL_TEXTURE_WRAP_S e GL_TEXTURE_WRAP_T.

A classe ViewerSingleton, trata-se de uma classe que aplica o padrão de projeto Singleton que gerencia os panoramas a serem visualizados, esta classe armazena o panorama selecionado pelo Usuário em GridViewActivity e carrega dinamicamente somente as imagens necessárias para a projeção, sendo assim, economiza recurso do dispositivo.

A classe ImageAdapter é utilizada para fazer a pré-visualização do panorama antes da projeção, esta classe é utilizada em GridViewerActivity.

O *enum* ViewType representa os tipos de projeções permitidas no aplicativo são elas, CUBICAL, representando a projeção cúbica, SPHERYCAL, representando a projeção esférica e ainda o tipo INVALID_TYPE que representa uma projeção inválida.

A classe FpsCalculator é responsável por calcular o *Frames Per Second* (FPS) e disponibilizá-lo como informação para o Usuário. Através do método calculateFps o FPS é calculado no método onDrawFrame de ViewerRenderer.

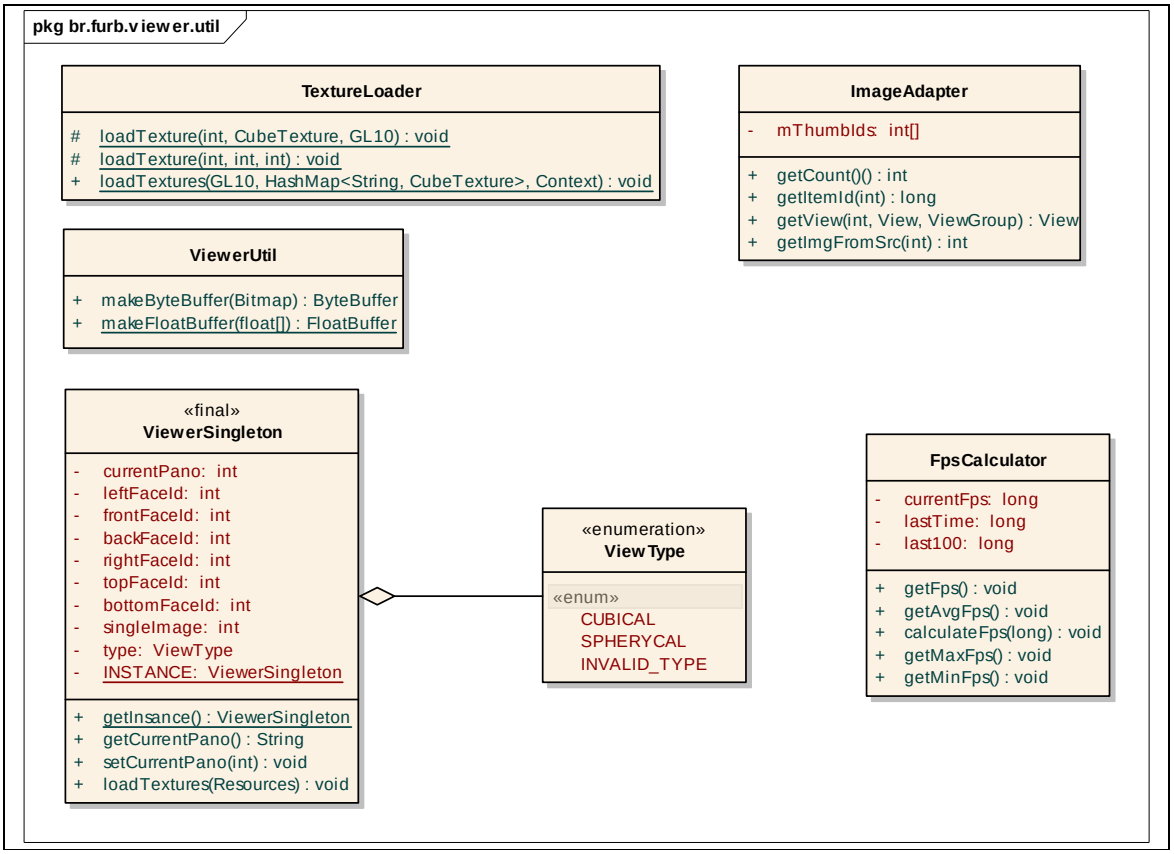


Figura 15 – Diagrama de classe do pacote `br.furb.viewer.util`

3.2.3 Diagrama de sequência

O diagramas de sequência (Figura 16) demonstra como o `Usuário` interage com o aplicativo relatado neste trabalho.

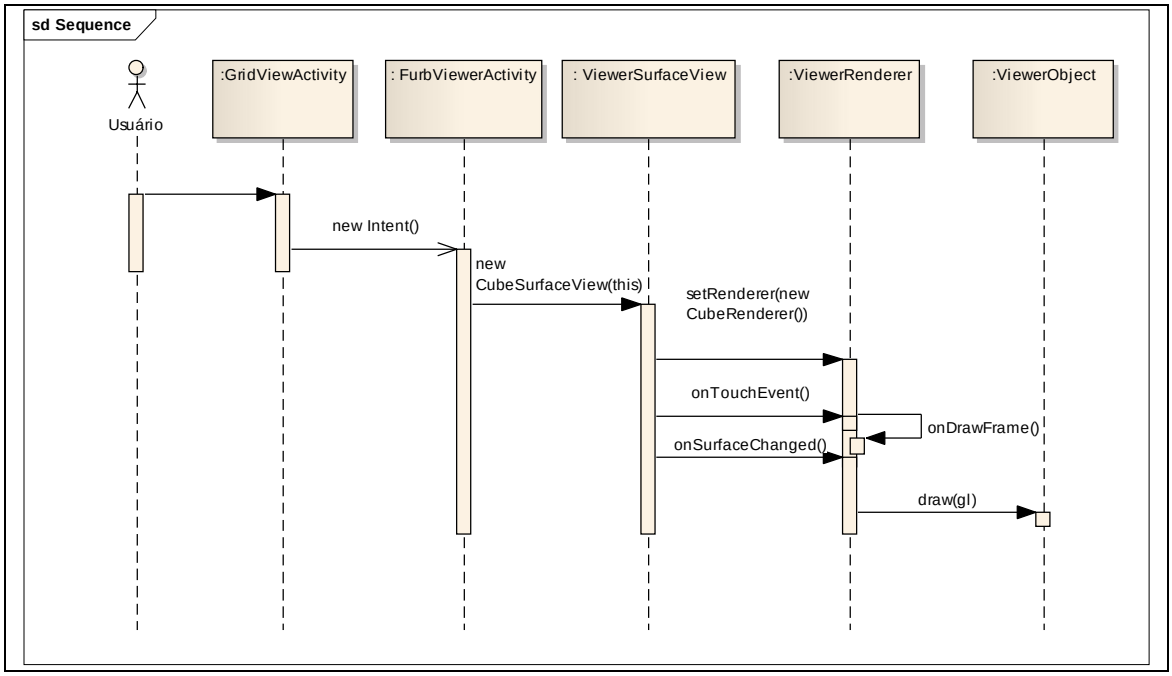


Figura 16 – Diagrama de sequência do aplicativo

3.3 IMPLEMENTAÇÃO

A seguir são mostradas as técnicas e ferramentas utilizadas e a operacionalidade da implementação, assim como detalhes de classes e rotinas.

3.3.1 Técnicas e ferramentas utilizadas

O desenvolvimento do aplicativo visualizador de imagens 360 graus, foi feito completamente na linguagem Java utilizando a API de desenvolvimento 2.2 do Android, também conhecida como Froyo, porém foi testado também na versão 2.3. O ambiente de desenvolvimento utilizado foi o Eclipse Indigo versão 3.7.2, juntamente com o *plugin* de desenvolvimento para o Android, o *Android Development Tools* (ADT), que possui vários recursos que facilitam o desenvolvimento, principalmente o depurador. O ADT permite que o desenvolvedor depure o código diretamente no dispositivo. Para execução e depuração do aplicativo foi utilizando também um dispositivo Samsung Galaxy. Para trabalhar com a projeção das imagens foi utilizado o OpenGL ES 1.0.

O depurador disponibilizado pelo ADT, permite que desenvolvedor execute e depure o código diretamente no dispositivo, aumentando a produtividade e a consistência dos resultados de testes. Possui algumas limitações, tais como o uso de *bluetooth* para se fazer a conexão com o dispositivo, algo que ainda não é permitido.

O Samsung Galaxy Fit S5670 possui processador de 600MHz com acelerador 3D, memória interna de 160mb e Android 2.3.4 Gingerbread. Possui ainda uma tela com resolução de 240x320 e 3.3 polegadas. A utilização de um dispositivo móvel foi particularmente necessária neste trabalho para realizar a implementação e os testes da funcionalidade de *zoom* e do acelerômetro.

3.3.2 O aplicativo FurbViewer

Abaixo são apresentados as principais partes do código fonte da aplicação, ordenados por importância e complexidade para o aplicativo.

3.3.2.1 A classe `ViewerRenderer`

O primeiro trecho de código a ser detalhado, é o método `onSurfaceCreated` da classe `ViewerRenderer`, este método é uma implementação da definição na interface `GLSurfaceView.Renderer`, chamado assim que a `View` do Android é criada, neste caso, assim que uma instância da classe `ViewerSurfaceView` é criada. Neste método são definidos os parâmetros iniciais do OpenGL, tais como, `glEnable (GL10.GL_DEPTH_TEST)` que ativa o teste de profundidade dos pixels, este método é seguido de `glDepthFunc (GL10.GL_LEQUAL)` que diz ao OpenGL que um pixel somente será desenhado se for menor ou igual ao pixel atual na coordenada x, y. Esta informação é importante no momento em que as imagens do panorama são unidas para dar a impressão de uma única imagem. É nesta parte da implementação também que o OpenGL é notificado que será utilizada texturas, do tipo 2D no aplicativo. O Quadro 5 apresenta o método `onSurfaceCreated` em sua totalidade.

```java
@Override
public void onSurfaceCreated(GL10 gl, EGLConfig config) {
    ViewerSingleton singleton = ViewerSingleton.getInstance();
    if(singleton.getViewType().equals(ViewType.CUBICAL)) {
        objectToDraw = new Cube(context, gl);
    } else if(singleton.getViewType().equals(ViewType.SPHERYCAL)) {
        objectToDraw = new Sphere(50, 50, .5f, 1.0f, gl, context);
    }
    // seta cor para Preto
    gl.glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    // clear value de profunidade
    gl.glClearDepthf(1.0f);
    // teste de profundidade dos pixels
    gl.glEnable(GL10.GL_DEPTH_TEST);
    //tipo de profundidade para visualizar o panorama
    gl.glDepthFunc(GL10.GL_LEQUAL);
    //sombreamento
    gl.glShadeModel(GL10.GL_SMOOTH);
    // Ativa textura
    gl.glEnable(GL10.GL_TEXTURE_2D);
    //Calcular o FPS
    fpsCalculator = new FpsCalculator();
    singleton.addFpsCalc(fpsCalculator);
}
```

Quadro 5 - Método `onSurfaceCreated`

Outro método relevante para o aplicativo nesta classe, é o método `onSurfaceChanged` que é um método executado após o método `onSurfaceCreated` ou após uma mudança no tamanho da tela. Um exemplo neste aplicativo é quando o dispositivo passa da horizontal para a vertical, ou vice-versa, neste caso o acelerômetro do aparelho detecta se a tela mudou e o método recebe as novas dimensões da tela. Neste aplicativo o modo de visualização é livre para escolha do `Usuário` no caso do método de interação ser o `Toque` caso seja o `Acelerômetro`, o modo de visualização fica fixo como "retrato". Este método também é uma implementação da interface `GLSurfaceView.Renderer`. Neste método são definidos os parâmetros iniciais do OpenGL, tais como, `glViewport`. Que neste caso é parametrizado para cobrir toda a tela, ele recebe por parâmetro a altura e a largura da tela, e passa estes parâmetros para o OpenGL. Sendo assim o aplicativo é visualizado em *full screen* no aparelho. A função mais importante deste método é chamar a função nativa do OpenGL `GLU.gluPerspective` que posiciona a câmera do OpenGL conforme a necessidade do objeto desenhado, por exemplo no caso da projeção cúbica a câmera é posicionada no centro do cubo. O Quadro 6 apresenta o método `onSurfaceChanged` completo.

```
@Override
public void onSurfaceChanged(GL10 gl, int width, int height) {
    // não permitir divisão por 0
    if (height == 0)
        height = 1;
    float aspect = (float) width / height;
    // Seta o viewport (área a ser visualizada) para cobrir toda a janela
    gl.glViewport(0, 0, width, height);
    // matriz de projeção
    gl.glMatrixMode(GL10.GL_PROJECTION);
    // Reseta a matriz de projeção
    gl.glLoadIdentity();
    // Posiciona a câmera no centro do objeto.
    GLU.gluPerspective(gl, 90, aspect, 0.01f, 100.f);
    // Seleciona model view matriz
    gl.glMatrixMode(GL10.GL_MODELVIEW);
    gl.glLoadIdentity();
    gl.glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    gl.glClearDepthf(1.0f);
}
```

Quadro 6 - Método `onSurfaceChanged`

Ainda na classe `ViewerRenderer` o método `onDrawFrame` é o método responsável por desenhar os objetos na tela a cada mudança de *frame*, apesar de o método não ser extenso, sua funcionalidade é importante para o sistema. Este método basicamente calcula o FPS e o coloca no *log*, limpa a tela, reinicia a matriz de transformação, faz a rotação e translação dos objetos conforme evento gerado pelo `Usuário`. Neste método a a translação do objeto é feita conforme a variável `z` é alterada, este variável muda toda vez que um evento de zoom é gerado. Já o método `glRotatef` tem a mesma funcionalidade do método da versão padrão do OpenGL este método rotaciona o objeto nos eixos x e y, através das variávies `angleX` e `angleY`. É este método que faz a chamada para o desenho do objeto. Maiores detalhes sobre este método são apresentados no Quadro 7.

```
@Override
public void onDrawFrame(GL10 gl) {
    fpsCalculator.calculateFps(System.currentTimeMillis());
    // Clear color e buffer da profundidade
    gl.glClear(GL10.GL_COLOR_BUFFER_BIT | GL10.GL_DEPTH_BUFFER_BIT);
    gl.glLoadIdentity(); // Reseta a matriz model view
    // Translada o cubo na tela
    gl.glTranslatef(0.0f, 0.0f, z);
    // Rotaciona o X conforme evento gerado pelo usuário do usuário
    gl.glRotatef(angleX, 1.0f, 0.0f, 0.0f);
    // Rotaciona o Z conforme evento gerado pelo do usuário
    gl.glRotatef(angleY, 0.0f, 1.0f, 0.0f);
    // Desenha o objeto
    objectToDraw.draw(gl);
}
```

Quadro 7 - Método `onDrawFrame`

3.3.2.2 A classe `ViewerSurfaceView`

A classe `ViewerSurfaceView` representa a tela que o `Usuário` interage. É nesta classe que os eventos de toque são tratados, esta classe é uma extensão da classe `GLSurfaceView` que por sua vez é uma implementação de uma `View` padrão do Android para o OpenGL. No método construtor desta classe são instanciados o `Renderer` da aplicação, um objeto da classe `ZoomListener` e um objeto da classe nativa do Android `ScaleGestureDetector`, para detectar eventos do tipo *zoom*.

O método `onTouchEvent` é responsável por tratar os eventos de toque por parte do `Usuário`, este método é herdado da classe `GLSurfaceView`. Primeiramente é chamado o método `onTouchEvent` de `ScaleGestureDetector`, que irá detectar se houve algum evento de multitoque que represente um *zoom in* ou *zoom out*. Se houver um evento do tipo *zoom* detectado, o método `onScale` da classe `ZoomListener` será chamado, que retornará um valor float entre 0.01 e 0.4 para ser utilizado no *zoom* da imagem (o valor inicial é 1.0). Quando o evento não é multitoque, ou seja somente um dedo tocou a tela, o aplicativo pega as novas coordenadas deste toque e posiciona o objeto conforme as novas coordenadas (x, y). O Quadro 8 apresenta o trecho de código que é executado quando um evento de toque que irá rotacionar a imagem acontece. O Quadro 9 demonstra o trecho de código que é executado quando o evento de multitoque é finalizado.

```
// o usuário moveu o dedo sob a tela
case MotionEvent.ACTION_MOVE: {
    final int pointerIndex = ev.findPointerIndex(mActivePointerId);
    final float x = ev.getX(pointerIndex);
    final float y = ev.getY(pointerIndex);
    if (!mScaleDetector.isInProgress()) {
        final float dx = x - mLastTouchX;
        final float dy = y - mLastTouchY;
        renderer.angleX += dy;
        renderer.angleY += dx;
    }
    mLastTouchX = x;
    mLastTouchY = y;
    break;
}
```

Quadro 8 – Tratamento de toque do `Usuário`

```
// evento toque no ponto "p" finalizado
case MotionEvent.ACTION_POINTER_UP: {
    final int pointerIndex = (ev.getAction() & MotionEvent.ACTION_POINTER_INDEX_MASK)
            >> MotionEvent.ACTION_POINTER_INDEX_SHIFT;
    final int pointerId = ev.getPointerId(pointerIndex);
    if (pointerId == mActivePointerId) {
        renderer.z = scaleListener.getmScaleFactor();
        final int newPointerIndex = pointerIndex == 0 ? 1 : 0;
        mLastTouchX = ev.getX(newPointerIndex);
        mLastTouchY = ev.getY(newPointerIndex);
        mActivePointerId = ev.getPointerId(newPointerIndex);
    }
    break;
}
```

Quadro 9 – Tratamento de multitoque do `Usuário`

3.3.2.3 A classe `Cube`

A classe `Cube` estende a classe `ViewerObject`, que é a classe que serve como base para outros objetos a serem desenhados na tela. Esta classe representa o objeto a ser utilizado na projeção cúbica. O principal processamento desta classe está no método `draw` que é um método sobrescrito da classe `ViewerObject`.

3.3.2.3.1 Desenho do cubo

Para fazer o desenho dos objetos são utilizados triângulos, para isso é passado um *array* com as coordenadas dos vértices para o OpenGL. Para tanto, a função `GL_VERTEX_ARRAY` do OpenGL é ativa, após isso, o OpenGL passa esperar um array com os vértices. Em seguida é chamada a função `glVertexPointer` que efetivamente passa os vértices do objeto através da variável `vertexBuffer` do tipo `FloatBuffer`. Também através deste método o OpenGL é informado que os vértices são definidos pelo pontos x, y e z do tipo `float`. O Quadro 10 apresenta a forma que os vértices dos objetos são carregados no OpenGL.

```
/**
 * Cria o Buffer com os vértices do objeto e ativa o modo GL_VERTEX_ARRAY
 * @param gl OpenGL padrão - 1.0
 */
protected void enableVertex(GL10 gl) {

    // ativa função de array de vértices
    gl.glEnableClientState(GL10.GL_VERTEX_ARRAY);
    // 3 pontos(x, y, z), tipo Float, Float Buffer.
    gl.glVertexPointer(3, GL10.GL_FLOAT, 0, vertexBuffer);

}
```

Quadro 10 – Carregar vértices do cubo

Após isto, as faces do cubo são desenhadas utilizando o método `glDrawArrays`, para cada face do cubo, existe uma chamada para este método. O primeiro parâmetro é o *`GL_TRIANGLE_STRIP`*, que é o modo de desenho a ser utilizado. O segundo indica a posição onde o OpenGL deve buscar os pontos no *array* e por último a quantidade de pontos a serem buscados a partir da posição informada no segundo parâmetro. Também neste método é feito o *`bind`* com a textura anteriormente carregada para cada face do cubo. O Quadro 11 mostra este processo.

```
@Override
public void draw(GL10 gl) {
    enableGLFunctions(gl);
    // cria Buffer dos vértices
    this.vertexBuffer = ViewerUtil.makeFloatBuffer(CubeConstants.cubeCoords);
    enableVertex(gl);
    //cria Buffer das texturas
    this.texBuffer = ViewerUtil.makeFloatBuffer(CubeConstants.cubeTexCoords);

    enableTextures(gl);

    // Desenha face frontal do cubo
    gl.glBindTexture(GL10.GL_TEXTURE_2D, textures.get("front").getId());
    gl.glDrawArrays(GL10.GL_TRIANGLE_STRIP, 0, 4);

    // Desenha face traseira do cubo
    gl.glBindTexture(GL10.GL_TEXTURE_2D, textures.get("back").getId());
    gl.glDrawArrays(GL10.GL_TRIANGLE_STRIP, 4, 4);

    // Desenha face esquerda do cubo
    gl.glBindTexture(GL10.GL_TEXTURE_2D, textures.get("left").getId());
    gl.glDrawArrays(GL10.GL_TRIANGLE_STRIP, 8, 4);

    // Desenha face direita do cubo
    gl.glBindTexture(GL10.GL_TEXTURE_2D, textures.get("right").getId());
    gl.glDrawArrays(GL10.GL_TRIANGLE_STRIP, 12, 4);

    // Desenha zênite do cubo
    gl.glBindTexture(GL10.GL_TEXTURE_2D, textures.get("top").getId());
    gl.glDrawArrays(GL10.GL_TRIANGLE_STRIP, 16, 4);

    // Desenha nadir do cubo
    gl.glBindTexture(GL10.GL_TEXTURE_2D, textures.get("bottom").getId());
    gl.glDrawArrays(GL10.GL_TRIANGLE_STRIP, 20, 4);

    disableGLFunctions(gl);
}
```

Quadro 11 – Desenho do cubo

3.3.2.4 Carregar textura

O processo de carregamento das texturas dos objetos, é semelhante ao carregamento dos vértices, há um *array* com as coordenadas das texturas do cubo que são passadas para o OpenGL, no caso da esfera os índices das texturas são obtidos no método que desenha a esfera.

Ainda no carregamento da textura, é importante salientar, que as imagens são carregadas conforme escolha, do `Usuário`, ou seja, somente são carregadas as imagens referentes ao panorama escolhido pelo `Usuário`. As imagens são carregadas dinamicamente, isto é possível através da classe `Resources` da API do Android. Esta classe possui o método

`getIdentifer` que retorna o identificador do recurso passado como parâmetro o que permite acesso dinâmico aos recursos declarado no arquivo `R.java`, que é um arquivo dinamicamente gerado pelo ADT com as referências de alguns recursos utilizados na aplicação, neste caso imagens. O Quadro 12 apresenta o carregamento dinâmico das imagens necessárias conforme o tipo de projeção.

```
public void loadTextures(Resources resources) {
    if (this.type.equals(ViewType.CUBICAL)) {
        this.frontFaceId = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano + "_f", null,
                null);
        this.backFaceId = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano + "_b", null,
                null);
        this.leftFaceId = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano + "_l", null,
                null);
        this.rightFaceId = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano + "_r", null,
                null);
        this.topFaceId = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano + "_u", null,
                null);
        this.bottomFaceId = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano + "_d", null,
                null);
    } else if(this.type.equals(ViewType.SPHERYCAL)) {
        this.singleImage = resources.getIdentifier(
                "br.furb.viewer:drawable/pano" + currentPano, null,
                null);
    }
}
```

Quadro 12 – Carregamento dinâmico das imagens do cubo

3.3.2.5 Desenho da esfera

O desenho da esfera é feito conforme algoritmo descrito por Smithwick e Mayank (2012, p. 65), comentado em 2.3.1. Para a projeção esférica é desenhada uma esfera perfeita. A resolução da esfera é feita utilizando o conceito de fatias horizontais e verticais, que são processadas em pares formando os limites da esfera. Cada interseção da fatia horizontal com a vertical, é subdivida em 2 triângulos sendo esta forma que OpenGL precisa para desenhar a esfera. Neste caso os *arrays* que armazenam os vértices e a texturas têm o dobro do tamanho das fatias horizontais, por exemplo, neste trabalho o número de fatias horizontais é 50, assim como o de verticais. Sendo assim, o tamanho do *array* é 102, o dobro do número de fatias,

mais 2, considerando o fato que os dois primeiros vértices são também os últimos, por isso estão duplicados. O *loop* externo trata das fatias horizontais começando pela pilha em -90° indo até 90°. Neste algoritmo é feita uma alusão à coordenadas geográficas para fazer o desenho, são utilizados como índices phi (Φ) e theta (Θ) para o *loop* de longitude (externo) e latitude (interno) respectivamente. Primeiramente no *loop* externo são gerados os limites específicos de cada linha, o fator `.5f` empurra tudo para baixo dos 90°, sem isso os valores iriam variar de 0° a 180°. O *loop* interno tem a matemática parecida com a do externo, porém definindo as fatias de 0° a 360°. Os quadros 13 e 14 apresentam o algoritmo de desenho da esfera.

```
private void buildSphere(int stacks,int slices, float radius, float squash, GL10 gl, Context context)
{
    float[] vertexData;
    float[] textData;

    int vIndex=0;               //indice de vértices
    int tIndex=0;               //índice de textura

    m_Scale=radius;
    m_Squash=squash;

    m_Stacks = stacks;        //50
                              //x
    m_Slices = slices;        //50

    //Vertices - dobro do tamanho, para cada "slice" são 2 triângulos
    vertexData = new float[ 3*((m_Slices*2+2) * m_Stacks)];

    textData = new float [2 * ((m_Slices*2+2) * (m_Stacks))];

    int phiIdx, thetaIdx;
    //Latitude - horizontal
    for(phiIdx=0; phiIdx < m_Stacks; phiIdx++)
    {
        // começa em - 90 graus e vai até 90.
        float phi0 = (float)Math.PI * ((float)(phiIdx)  * (1.0f/(float)(m_Stacks)) - 0.5f);
        // trata o próximo
        float phi1 = (float)Math.PI * ((float)(phiIdx+1) * (1.0f/(float)(m_Stacks)) - 0.5f);
        float cosPhi0 = (float)Math.cos(phi0);
        float sinPhi0 = (float)Math.sin(phi0);
        float cosPhi1 = (float)Math.cos(phi1);
        float sinPhi1 = (float)Math.sin(phi1);
        float cosTheta, sinTheta;
```

Quadro 13 – Primeira parte do algoritmo que constrói a esfera

```
            //longitude - vertical
            for(thetaIdx=0; thetaIdx < m_Slices; thetaIdx++)
            {
                float theta = (float) (2.0f*(float)Math.PI * ((float)thetaIdx) * (1.0/(float)(m_Slices-1)));
                cosTheta = (float)Math.cos(theta);
                sinTheta = (float)Math.sin(theta);
                vertexData[vIndex]   = m_Scale*cosPhi0*cosTheta;
                vertexData[vIndex+1] = m_Scale*(sinPhi0*m_Squash);
                vertexData[vIndex+2] = m_Scale*(cosPhi0*sinTheta);
                vertexData[vIndex+3] = m_Scale*cosPhi1*cosTheta;
                vertexData[vIndex+4] = m_Scale*(sinPhi1*m_Squash);
                vertexData[vIndex+5] = m_Scale*(cosPhi1*sinTheta);
                float texX = (float)thetaIdx * (1.0f/(float)(m_Slices-1));
                textData [tIndex + 0] = texX;
                textData [tIndex + 1] = (float)(phiIdx+0) * (1.0f/(float)(m_Stacks));
                textData [tIndex + 2] = texX;
                textData [tIndex + 3] = (float)(phiIdx+1) * (1.0f/(float)(m_Stacks));
                vIndex+=2*3;
                tIndex+= 2*2;
                vertexData[vIndex+0] = vertexData[vIndex+3] = vertexData[vIndex-3];
                vertexData[vIndex+1] = vertexData[vIndex+4] = vertexData[vIndex-2];
                vertexData[vIndex+2] = vertexData[vIndex+5] = vertexData[vIndex-1];
                textData [tIndex + 0] = textData [tIndex + 2] = textData [tIndex -2];
                textData [tIndex + 1] = textData [tIndex + 3] = textData [tIndex -1];

            }
        }

        vertexBuffer = ViewerUtil.makeFloatBuffer(vertexData);
        texBuffer = ViewerUtil.makeFloatBuffer(textData);
    }
```

Quadro 14 – Segunda parte do algoritmo que constrói a esfera

3.3.3 Operacionalidade do aplicativo

A operacionalidade do aplicativo será apresentada através da simulação dos casos de uso do sistema previamente apresentados. Serão simulados os passos necessários que o `Usuário` deverá seguir para interagir com o sistema. Nesta seção serão demonstradas imagens do aplicativo funcionando no Galaxy S Fit.

A tela inicial do sistema possui basicamente algumas informações sobre o trabalho e um botão `Entrar` que ao ser clicado direciona para a tela de escolha de imagens panorâmicas. A Figura 17 apresenta a tela inicial do sistema.

Figura 17 – Tela inicial do aplicativo

A segunda tela apresentada ao `Usuário`, apresenta miniaturas das imagens disponíveis para visualização panorâmica, assim como permite ao `Usuário` escolher em qual projeção deseja visualizar a imagem e qual o modo de interação a ser utilizado. Primeiramente o `Usuário` deve escolher em qual projeção deseja visualizar o panorama, por padrão, o sistema deixa previamente selecionada a projeção cúbica. A Figura 18 demonstra como o `Usuário` deve fazer esta escolha.

Figura 18 – Escolha da projeção

Com a projeção escolhida, o `Usuário` deve escolher a forma de interação com o aplicativo (Toque ou Acelerômetro), A Figura 19 apresenta a simulação desta escolha.

Figura 19 – Escolher modo de interação

Após escolher o modo de projeção desejado o `Usuário` deve escolher a imagem que deseja visualizar o panorama. A Figura 20 demonstra o `Usuário` escolhendo a imagem.

Figura 20 – Escolher imagem para projeção

Após a escolha, o sistema irá apresentar a imagem na projeção desejada, a visualização inicial fica no centro da imagem, porém o `Usuário` pode girar a imagem em 360 graus tanto no eixo x, quanto no eixo y. As Figuras 21 e 22 apresentam giros no eixo x e y respectivamente.

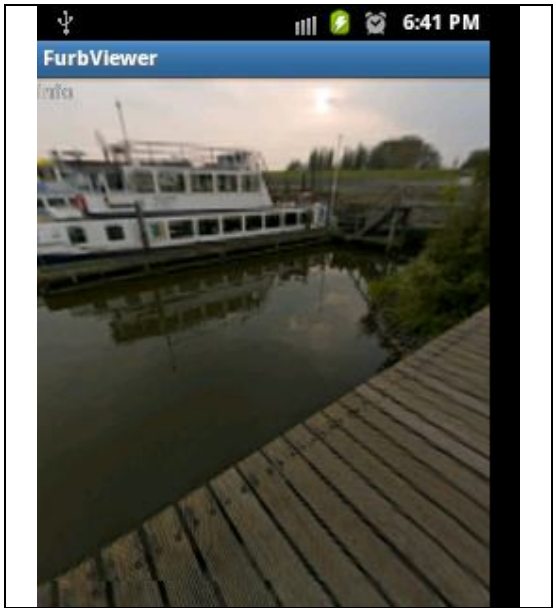


Figura 21 – Rotação no eixo x

Figura 22 – Rotação no eixo y

Ainda nesta tela o `Usuário` pode aplicar *zoom* na imagem, tanto para aproximar quanto para distanciar, para fazer o *zoom in*, por exemplo, o `Usuário` necessita tocar a tela com os dois dedos juntos e separá-los. A Figura 23 apresenta uma imagem com o *zoom in* aplicado.

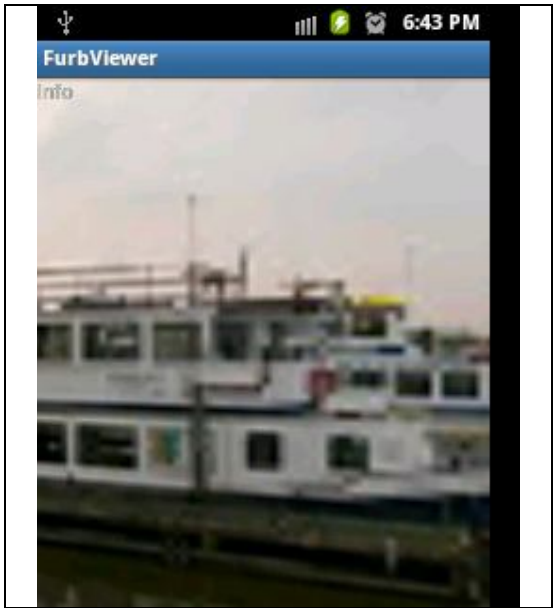


Figura 23 – *Zoom in* no na imagem

No canto superior esquerdo da tela é possível visualizar um *label* `info` que ao ser clicado apresenta as informações do FPS da visualização da imagem. A Figura 24 uma imagem com as informações do FPS.

Figura 24 – Informações do FPS

3.4 RESULTADOS E DISCUSSÃO

Algumas dificuldades foram encontradas no momento do desenvolvimento do aplicativo. No momento do desenvolvimento do carregamento da textura no aplicativo foi encontrado um problema ao carregar a imagem na face do cubo. A imagem estava adquirindo um aspecto triangular, os triângulos (`GL_TRIANGLE_STRIP`) utilizados para fazer o desenho, estavam claramente visíveis. Para fazer o carregamento da textura estava sendo utilizando o método `texImage2D` da classe `GLUtils` que é uma classe nativa do Android utilizada para fazer a ligação do Android com o OpenGL ES. Para fazer o carregamento das texturas, passou-se a utilizar diretamente o OpenGL através do método `glTexImage2D`, que permite informar ao OpenGL alguns parâmetros da textura, como altura, largura e formato interno. Após esta alteração a imagem passou a ser projetada da forma esperada. O Quadro 15 apresenta o modo antigo de carregar as imagens, já o Quadro 16 apresenta a forma atual.

```
public void loadTexture(GL10 gl, Context context) {
   //6 faces do cubo
    gl.glGenTextures(6, array, 0);

   for (int face = 0; face < numFaces; face++) {
       gl.glBindTexture(GL10.GL_TEXTURE_2D, textureIDs[face]);   // Bind to texture ID
       gl.glTexParameterf(GLES10.GL_TEXTURE_2D, GLES10.GL_TEXTURE_MIN_FILTER, GLES10.GL_NEAREST);
       gl.glTexParameterf(GLES10.GL_TEXTURE_2D, GLES10.GL_TEXTURE_MAG_FILTER, GLES10.GL_LINEAR);
       GLUtils.texImage2D(GL10.GL_TEXTURE_2D, 0, bitmap[face], 0);
       bitmap[face].recycle();
 }
```

Quadro 15 – Carregamento antigo das texturas do cubo.

```
protected static void loadTexture(int texture, int type, int width,
        int height, ByteBuffer bb, GL10 gl) {
    gl.glBindTexture(type, texture);
    gl.glTexImage2D(type, 0, GL10.GL_RGBA, width, height, 0, GL10.GL_RGBA,
            GL10.GL_UNSIGNED_BYTE, bb);
    gl.glTexParameterf(GLES10.GL_TEXTURE_2D, GLES10.GL_TEXTURE_MIN_FILTER,
            GLES10.GL_LINEAR);
    gl.glTexParameterf(GLES10.GL_TEXTURE_2D, GLES10.GL_TEXTURE_MAG_FILTER,
            GLES10.GL_LINEAR);

}
```

Quadro 16 – Carregamento atual das texturas do cubo.

Outro ponto importante no momento do carregamento das texturas foram as arestas do cubo, na montagem das imagens, inicialmente por padrão o OpenGL mantém as arestas visíveis. Para resolver este problema os parâmetros de textura do OpenGL,

`GL_TEXTURE_WRAP_S` e `GLES10.GL_TEXTURE_WRAP_T`, foram configurados com a opção `GL_CLAMP_TO_EDGE`, com isso os *pixels* da imagem sobrepõem as arestas do cubo, após isso, as 6 imagens parecem ser uma só, possibilitando a visualização de um panorama.

Para realizar os testes do aplicativo foram utilizados tanto o simulador quanto o Samsung Galaxy Fit. Para medir o desempenho do aplicativo foram utilizados FPS e quantidade de memória *heap* utilizada pelo aplicativo. Para realizar a medição, foi implementando na classe `ViewerRenderer`, método `onDrawFrame`, o cálculo do FPS, que é calculado a cada chamada deste método.

Para realizar os testes foram utilizadas diferentes imagens, com diferentes resoluções nas duas projeções implementadas neste trabalho, no intuito de testar o comportamento do aplicativo nestes diferentes cenários. A primeira projeção aqui demonstrada é a projeção esférica, posteriormente serão apresentados os testes na projeção cúbica. Por fim uma comparação entre as projeções é apresentada.

3.4.1 Projeção Esférica

A projeção esférica é feita com somente uma imagem, por isso é mais fácil de encontrar as imagens na internet. Para fazer os testes foram utilizadas imagens com as seguintes resoluções: 2.000x1.000, 1.280x640, 1.024x512, 500x250 e 320x160. A Tabela 1 apresenta os testes destas imagens utilizando o Samsung Galaxy Fit.

Tabela 1 - Teste de FPS na projeção esférica

Resolução da imagem	FPS (média)	Total de Pixels
320x160	93	51.200
500x250	90	125.000
1.024x512	91	524.888
1.280x640	91	819.200
2.000x1000	88	2.000.000

Apesar de haver uma grande variação na resolução das imagens o FPS se manteve próximo da casa dos 90. O que demonstra que o aplicativo está preparado para receber imagens em diferentes resoluções sem diminuir a qualidade da visualização.

Ainda utilizando o Samsung Galaxy Fit, foi testado a alocação de memória *heap*, para cada uma das imagens acima. A Tabela 2 apresenta os resultados.

Tabela 2 – Memória *heap* alocada na projeção esférica

Resolução da imagem	Memória heap (mb)	Total de Pixels
320x160	2.873	51.200
500x250	2.873	125.000
1.024x512	2.875	524.888
1.280x640	2.876	819.200
2.000x1000	2.847	2.000.000

A utilização da memória manteve-se constante, independentemente da resolução da imagem utilizada para projeção. A taxa de ocupação da memória *heap* foi em média 49%.

Os mesmos testes foram refeitos, porém, utilizando o simulador da plataforma Android. No caso do FPS, é possível perceber uma queda brusca comparando o simulador com o dispositivo, o que demonstra que o simulador precisa de melhorias quando o aplicativo possui processamento gráfico. A Tabela 3 apresenta os resultados dos testes envolvendo FPS no simulador.

Tabela 3 – Teste de FPS na projeção esférica (simulador)

Resolução da imagem	FPS (média)	Total de Pixels
320x160	7	51.200
500x250	5	125.000
1.024x512	6	524.888
1.280x640	5	819.200
2.000x1000	5	2.000.000

A Tabela 4 apresenta o teste de memória utilizando o simulador.

Tabela 4 – Memória *heap* alocada na projeção esférica (simulador)

Resolução da imagem	Memória heap (mb)	Total de Pixels
320x160	2.935	51.200
500x250	2.915	125.000
1.024x512	2.956	524.888
1.280x640	2.955	819.200
2.000x1000	2.978	2.000.000

Nos testes envolvendo a memória *heap*, o simulador obteve um desempenho muito parecido com o do dispositivo. A alocação de memória subiu, porém, muito pouco, não afetando o desempenho do aplicativo.

3.4.2 Projeção Cúbica

Para a projeção cúbica, são utilizadas seis imagens, a simetria destas imagens é um processo trabalhoso. Por isso imagens cúbicas não são facilmente encontradas. Neste trabalho

foi utilizado um utilitário para converter as imagens equiretangulares (esféricas) para as imagens cúbicas. As imagens utilizadas foram as mesmas utilizadas na projeção esférica, o que muda é que cada uma das resoluções nas Tabelas 5 e 6, referem-se as 6 imagens. Os resultados dos testes de FPS da projeção cúbica utilizando o dispositivo Galaxy Fit estão no Tabela 5.

Tabela 5 – Teste de FPS na projeção cúbica

Resolução da imagem	FPS (média)	Total de Pixels
103x103	91	63.654
161x161	91	155.526
327x327	90	641.574
409x409	90	1.003.686
638x638	89	2.442.264

Com estes testes, foi possível averiguar que independente da resolução, o FPS se mantém num valor próximo e que não afetou o desempenho do aplicativo. Ainda utilizando o Galaxy Fit, foram feitos testes de uso de memória *heap* apresentados na Tabela 6.

Tabela 6 – Memória *heap* alocada na projeção cúbica

Resolução da imagem	Memória heap (mb)	Total de Pixels
103x103	2.958	63.654
161x161	2.982	155.526
327x327	3.048	641.574
409x409	3.051	1.003.686
638x638	3.107	2.442.264

Neste teste, foi possível verificar que a alocação de memória subiu pouco conforme a resolução da imagem foi aumentando, ocupando cerca de 50% (em média) do espaço padrão disponibilizado para o aplicativo.

Assim como na projeção esférica, os testes foram refeitos utilizando o simulador e novamente houve uma queda brusca na taxa de FPS do aplicativo. A Tabela 7 apresenta o teste realizado na projeção cúbica, utilizando o simulador.

Tabela 7 – Teste de FPS na projeção cúbica (simulador)

Resolução da imagem	FPS (média)	Total de Pixels
103x103	9	63.654
161x161	8	155.526
327x327	8	641.574
409x409	8	1.003.686
638x638	7	2.442.264

Assim como na projeção esférica, os testes do FPS no simulador na projeção cúbica obtiveram valores muito abaixo dos obtidos no dispositivo. A Tabela 8 apresenta o teste de memória utilizando o simulador.

Tabela 8 – Memória *heap* alocada na projeção cúbica (simulador)

Resolução da imagem	Memória heap (mb)	Total de Pixels
103x103	2.664	63.654
161x161	2.678	155.526
327x327	2.746	641.574
409x409	2.854	1.003.686
638x638	3.155	2.442.264

No teste de memória alocada os valores não foram divergentes em relação ao teste no dispositivo. A alocação de memória subiu conforme a resolução da imagem aumentou, porém, subiu pouco, de uma forma que não afeta o desempenho do aplicativo.

3.4.3 Comparação entre as projeções

Com estes testes, foi possível medir o desempenho do aplicativo nas duas projeções utilizadas. As duas projeções obtiveram resultados muitos semelhantes tanto no teste do FPS quanto na alocação de memória *heap*. A projeção esférica apresenta uma ligeira vantagem na alocação de memória, porém, são valores irrisórios que praticamente não influenciam no desempenho do aplicativo. Conforme os testes realizados, imagens com uma resolução grande não influenciam sensivelmente o desempenho do aplicativo. Nestes testes também foi possível verificar que o simulador do Android tem uma carência grande quando é necessário simular um processamento gráfico. A taxa de FPS caiu bruscamente nas duas projeções quando o simulador foi utilizado. Por isso, o uso de um dispositivo para realizar os testes é altamente necessário. Os eixos horizontais dos gráficos representam primeiro a resolução da imagem na projeção cúbica seguido pela resolução equivalente na projeção esférica. Já o eixo vertical representa o FPS. A Figura 25 apresenta o gráfico comparando a taxa FPS das projeções, quando foi o utilizando o Galaxy Fit para realizar os testes. A Figura 26 demonstra a comparação entre as duas projeções no teste de memória, também utilizando o Galaxy Fit.

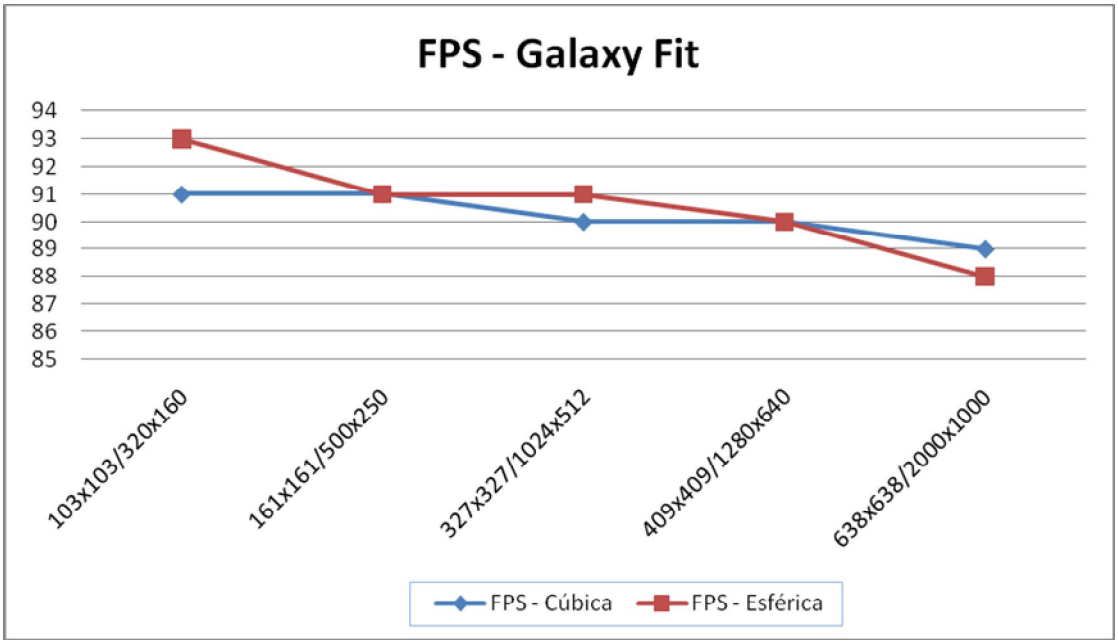


Figura 25 – Comparação da taxa de FPS das duas projeções

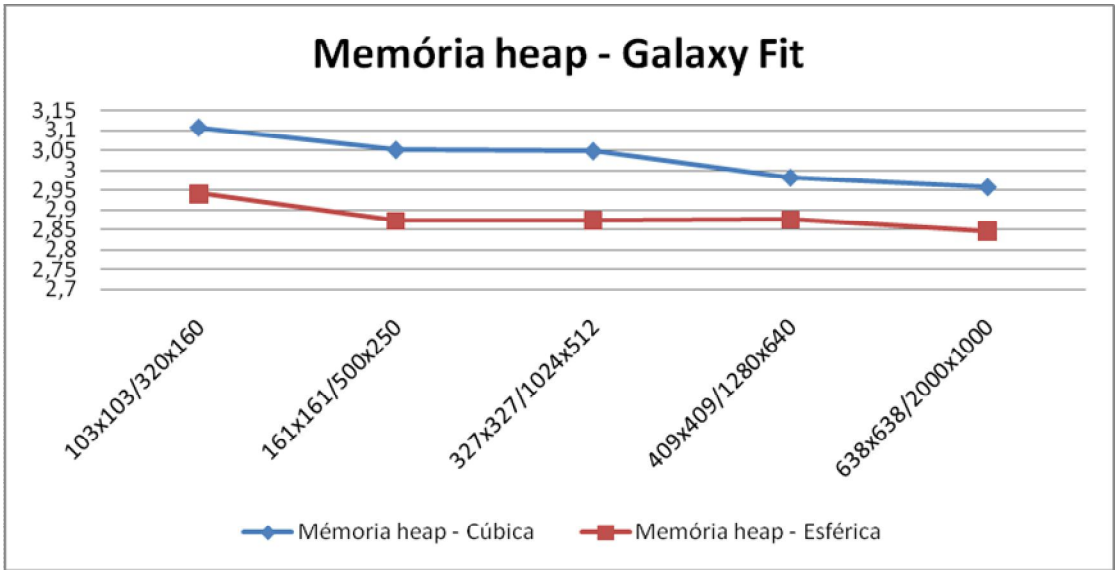


Figura 26 – Comparação de alocação de memória das duas projeções

Por último a Figura 27 apresenta um exemplo de uma das imagens utilizadas no trabalho projetada.

Figura 27 – Exemplo de imagem projetada

3.4.4 Comparação com os trabalhos correlatos

Neste seção é demonstrado uma comparação entre os trabalhos correlatos previamente apresentados e o aplicativo desenvolvido neste trabalho, denominado de `FurbViewer`. Para fazer esta comparação foram utilizados os fatores, quantidade projeções, suporte a captura de imagens, plataforma móvel e se possui carregamento de imagens. O Quadro 17 apresenta a comparação realizada.

Aplicativo	Projeções	Captura de Imagens	Plataforma Móvel	Carregar imagens
FurbViewer	Cúbica, esférica	não	sim	não
PTViewer	Cilíndrica, esférica	não	não	sim
PhotoAF	Cilíndrica	sim	sim	sim
Web Rotate	?	sim	não	sim

Quadro 17 – Comparação com os trabalhos correlatos

É possível verificar que o ponto forte deste trabalho são as duas projeções utilizadas, principalmente a projeção cúbica, que proporciona uma imagem com distorção mínima. Porém, fica abaixo dos trabalhos correlatos, quando o assunto é carregamento de imagens de forma dinâmica, já que este trabalho não contemplou esta parte. Quanto ao trabalho correlato Web Rotate 360, não foi possível descobrir qual projeção é utilizada pois é uma ferramenta comercial e não disponibiliza este tipo de informação.

4 CONCLUSÕES

Este trabalho apresentou um aplicativo no qual é possível visualizar imagens em 360 graus na plataforma Android. O aplicativo foi baseado em duas das projeções estudadas, cúbica e esférica. O aplicativo desenvolvido neste trabalho, proporciona ainda, interação como a imagem projetada na projeção escolhida utilizando dos recursos de toque na tela do dispositivo e também utilizando o acelerômetro do dispositivo.

Neste trabalho foi possível explorar as funcionalidades da plataforma Android, focando principalmente na parte gráfica da mesma. Foi utilizado o OpenGL ES na versão 1.0 para realizar as projeções das imagens na plataforma, o OpenGL mostrou-se estável, a escolha da versão do OpenGL foi baseada no suporte da mesma no simulador, pois a versão 2.0 não possui suporte no simulador. Quanto ao simulador, para testes envolvendo processamento gráfico, o desempenho ainda é muito abaixo de um dispositivo móvel. Por isso, optou-se por usar um dispositivo móvel para realizar os testes. Além disso, o uso de um dispositivo foi necessário devido as funcionalidades de *zoom*, que não são possíveis no simulador.

Este aplicativo apresenta funcionalidades semelhantes as dos trabalhos correlatos, porém, com o diferencial de ser na plataforma Android. Outro diferencial do trabalho é a possibilidade de campo de visão de 360 graus tanto no eixo horizontal quanto no vertical. O que foi possível após o estudo das duas projeções apresentadas que permitem este tipo de visualização. Entretanto, comparando com os trabalhos correlatos, é possível verificar que este trabalho não contempla a captura de imagens para a visualização panorâmica.

Por fim, é importante salientar que todos os objetivos definidos foram atingidos, tanto os requisitos funcionais, quanto o objetivo de explorar a computação gráfica utilizando um dispositivo móvel. Assim como, adicionalmente aos requisitos apresentados, foi feito a interação com a imagem utilizando o acelerômetro do dispositivo. Os objetivos relacionados à interação com o aplicativo foram alcançados através do uso da biblioteca padrão do Android. Tais como a utilização dos recursos de multitoque do dispositivo utilizado para testes no trabalho.

4.1 EXTENSÕES

Durante o desenvolvimento deste trabalho foram identificadas algumas extensões que podem ser aplicadas no mesmo.

A primeira e principal extensão, seria incluir a funcionalidade de captura de imagens, para isso seria necessário o estudo de técnicas de capturas e formas de conversão. Seria necessário também estudar novas projeções para averiguar qual melhor se adapta as limitações da câmera dos dispositivos.

Neste trabalho foram implementadas duas projeções, poderiam ainda ser estudas as demais projeções e implementadas neste trabalho. A partir destas projeções, poderia ser criado um *framework*, para desenvolver aplicativos que possuam visualização panorâmica. Além disso, algumas funcionalidades do sistema poderiam ser desenvolvidas utilizando bibliotecas C/C++, no intuito de testar o desempenho do aplicativo.

Uma outra possibilidade de melhoria, seria o carregamento de imagens de forma dinâmica, buscando as imagens de algum repositório específico. Como neste trabalho o foco era o desenvolvimento da parte gráfica, as imagens têm o seu carregamento de forma fixa.

REFERÊNCIAS BIBLIOGRÁFICAS

ANDROID MARKET. **Photoaf 3D**. [S.l.], 2011. Disponível em: <https://market.android.com/details?id=obg1.Photaf>. Acesso em: 06 set. 2011.

APPLE. **QuickTime VR questions and answers**. [S.l.], 2011. Disponível em: <http://support.apple.com/kb/TA21662?viewlocale=en_US>. Acesso em: 25 out. 2011.

AUTODESK. **AutoCAD**. [S.l.], 2011. Disponível em: <http://docs.autodesk.com/ACD/2011/ENU/landing.html>. Acesso em: 25 out. 2011.

AZEVEDO, Eduardo; CONCI, Aura. **Computação gráfica:** teoria e prática. Rio de Janeiro: Elsevier, 2003.

BAEZ, Javier; ÑAUÑAY, Miguel. **PanoramaGL library**. [S.l.], 2010. Disponível em: <http://code.google.com/p/panoramagl-android/>. Acesso em: 25 out. 2011.

BENTLEY. **MicroStation**. [S.l.], 2011. Disponível em: <http://www.bentley.com/pt-BR/Products/MicroStation>. Acesso em: 25 out. 2011.

CARVALHO, Adelson S. et al. Fotos panorâmicas educativas: o uso do Hugin como instrumento de criação e construção de significados artísticos-visuais. In: CICLO DE PALESTRAS DA RENOTE, 17., 2009, Porto Alegre. **Anais...** Porto Alegre: UFGRS, 2009. p. 1-8.

DERSCH, Helmut. **PTViewer**. [S.l.], 2001. Disponível em: <http://webuser.fh-furtwangen.de/~dersch/PTVJ/doc.html>. Acesso em: 06 set. 2011.

GOOGLE. **What is Android**. [S.l.], 2011a. Disponível em: <http://developer.android.com/guide/basics/what-is-android.html>. Acesso em: 05 set. 2011.

_____. **3D with OpenGL**. [S.l.], 2011b. Disponível em: <http://developer.android.com/guide/topics/graphics/opengl.html>. Acesso em: 06 set. 2011.

JACOBS, Corinna. **Interactive panoramas**. Nova York: Springer, 2004.

ITEN JR., Jorge Luis. **Visualizador de imagens 360 graus interativas na plataforma Android**: documentação. Blumenau, 2012. Disponível em: <http://www.inf.furb.br/gcg/gcg/download/TCC2012-1-17-JorgeLItenJ_documentacao.zip>. Acesso em: 12 jun. 2012.

KOMATINENI, Satya; MACLEAN, Dave; HASHIMI, Sayed. **Pro Android 3**. Nova York: Springer, 2011.

MCHUGH, Sean. **Projeção**. [S.l.], 2010. Disponível em: <http://www.cambridgeincolour.com/tutorials/image-projections.htm>. Acesso em: 03 set. 2011.

MEIER, Reto. **Professional Android 2 application development**. Indianapolis: Wiley, 2010.

PANOTOOLS. **Projeção cúbica.** [S.l.], 2007. Disponível em: <http://wiki.panotools.org/Cubic_Projection>. Acesso em: 17 set. 2011.

_____. **Projeção cilíndrica**. [S.l.], 2008a. Disponível em: <http://wiki.panotools.org/Cylindrical_Projection>. Acesso em: 17 set. 2011.

_____. P**rojeção equirectangular**. [S.l.], 2008b. Disponível em: <http://wiki.panotools.org/Equirectangular>. Acesso em: 17 set. 2011.

_____. **Projeção fisheye**. [S.l.], 2008c. Disponível em: <http://wiki.panotools.org/Fisheye_Projection>. Acesso em: 17 set. 2011.

PEREIRA, Lucio C. O.; SILVA, Michel L. da. **Android para desenvolvedores**. Rio de Janeiro: Brasport, 2009.

PTVIEWER. **PTViewer 2.8**. [S.l.], 2006. Disponível em: <http://www.fsoft.it/panorama/ptviewer.htm>. Acesso em: 03 set. 2011.

PULLI, Kari et al. **Mobile 3D graphics:** with OpenGL ES and M3G. Burlington: Elsevier, 2008.

RAY, Sidney F. **Scientific photography and applied imaging.** Woburn: Focal Press, 2002.

SMITHWICK, Mike; MAYANK, Verma. **Pro OpenGL ES for Android**. Nova York: Apress, 2012.

VALENTE, Augusto C. **Análise comparativa de compressão de imagens fisheye, retilíneas e panorâmicas**. 2010. 49 f. Monografia (Bacharelado em Engenharia Elétrica) - Universidade de Brasília, Brasília, 2010.

WEB ROTATATE 360 LLC. **About web rotate 360**. [S.l.], 2011. Disponível em: <http://www.webrotate360.com/360_product_viewer_download/Readme.pdf>. Acesso em: 06 set. 2011.